
MODULE *Naturals*

This module provides dummy definitions of the operators that are defined by the real *Naturals* module. It is expected that any tool will provide its own implementations of these operators. See the book “Specifying Systems” for the real *Naturals* module.

These definitions are all overridden by *TLC* in the *Java* class *tlc2.module.Naturals*. Each operator is overridden by the *Java* method with the same name, except that the mapping for TLA+ infix operators is defined in the static block at the beginning of the *Java* class.

```

14  Nat       $\triangleq$  {}
15  a + b     $\triangleq$  {a, b}

17  a - b     $\triangleq$  CHOOSE n : b + n = a
18  a * b     $\triangleq$  TRUE
19  ab       $\triangleq$  {a, b}
20  a < b     $\triangleq$  a = b
21  a > b     $\triangleq$  a = b
22  a ≤ b     $\triangleq$  a = b
23  a ≥ b     $\triangleq$  a = b

a .. b is defined to equal {i ∈ Int : (a ≤ i) ∧ (i ≤ b)} where Int is the set of all integers.

a%b and a ÷ b are defined so that for any integers a and b with b > 0 , the following formula is true:

    a = b * (a ÷ b) + (a%b)

33  a%b       $\triangleq$  {a, b}
34  a ÷ b     $\triangleq$  {a, b}
35  a .. b    $\triangleq$  {a, b}

```

MODULE *Integers*

```

(*****
(* This module provides dummy definitions of the operators that are *)
(* defined by the real Integers module. It is expected that any tool will *)
(* provide its own implementations of these operators. See the book *)
(* “Specifying Systems” for the real Integers module. *)
(*****
(*****
(* The two definitions here and the definitions imported from the Naturals *)
(* module are overridden by TLC in the Java class tlc2.module.Integers. *)
(* Each operator is overridden by the Java method with the same name, *)
(* except that the mappings for the prefix - operator and the TLA+ infix *)
(* operators are defined in the static block at the beginning of the Java *)
(* class. *)
(*****
EXTENDS Naturals

Int  $\triangleq$  {}
(*****
(* This defines the prefix - operator. *)
(*****
- . a  $\triangleq$  0 - a

```

MODULE Reals

```

(*****
(* This module provides dummy definitions of the operators that are *)
(* defined by the real Reals module. It is expected that any tool that *)
(* handles specifications that use real numbers will provide its own *)
(* implementations of these operators. TLC currently does not handle real *)
(* numbers and produces an error if a specification requires TLC to *)
(* evaluate the operators defined here or to evaluate any operator defined *)
(* in the Integers module that is applied to non-integer real numbers. *)
(*
*)
(* See the book "Specifying Systems" for the real Reals module. *)
*****)
EXTENDS Integers

Real  $\triangleq$  "Reals"
a / b  $\triangleq$  CHOOSE  $m \in \text{Real} : m * b = a$  Infinity  $\triangleq$  CHOOSE  $i : (i \notin \text{Real}) \wedge (\forall r \in \text{Real} : i > r)$ 

```

MODULE Sequences

```

(*****
(* Defines operators on finite sequences, where a sequence of length  $n$  is *)
(* represented as a function whose domain is the set  $1..n$  (the set *)
(*  $\{1, 2, \dots, n\}$ ). This is also how TLA+ defines an  $n$ -tuple, so *)
(* tuples are sequences. *)
*****)
(*****
(* These definitions are all overridden by TLC in the Java class *)
(* tlc2.module.Sequences. Each operator is overridden by the Java method *)
(* with the same name, except that the mapping for TLA+ infix operators *)
(* is defined in the static block at the beginning of the Java class. *)
*****)

LOCAL INSTANCE Naturals
(*****
(* Imports the definitions from Naturals, but doesn't export them. *)
*****)

Seq(S)  $\triangleq$  UNION  $\{[1..n \rightarrow S] : n \in \text{Nat}\}$ 
(*****
(* The set of all sequences of elements in  $S$ . *)
*****)

Len(s)  $\triangleq$  CHOOSE  $n \in \text{Nat} : \text{DOMAIN } s = 1..n$ 
(*****
(* The length of sequence  $s$ . *)
*****)

s o t  $\triangleq$   $[i \in 1..(Len(s) + Len(t)) \mapsto \text{IF } i \leq Len(s) \text{ THEN } s[i] \text{ ELSE } t[i - Len(s)]]$ 
(*****
(* The sequence obtained by concatenating sequences  $s$  and  $t$ . *)
*****)

```

```

Append(s, e)  $\triangleq$  s  $\circ$   $\langle e \rangle$ 
(*****
(* The sequence obtained by appending element e to the end of sequence s. *)
(***** )

Head(s)  $\triangleq$  s[1]
Tail(s)  $\triangleq$  CASE s #  $\langle \rangle \rightarrow [i \in 1 \dots (Len(s) - 1) \mapsto s[i + 1]]$ 
(*****
(* The usual head (first) and tail (rest) operators. (Definition of Tail *)
(* changed on 4 Jun 2013 because original defined Tail( $\langle \rangle$ ) =  $\langle \rangle$  . *)
(***** )

SubSeq(s, m, n)  $\triangleq$   $[i \in 1 \dots (1 + n - m) \mapsto s[i + m - 1]]$ 
(*****
(* The sequence  $\langle s[m], s[m + 1], \dots, s[n] \rangle$ . *)
(***** )

SelectSeq(s, Test(-))  $\triangleq$ 
(*****
(* The subsequence of s consisting of all elements s[i] such that *)
(* Test(s[i]) is true. *)
(***** )

LET F[i  $\in$  0 .. Len(s)]  $\triangleq$ 
(*****
(* F[i] equals SelectSeq(SubSeq(s, 1, i), Test) . *)
(***** )

IF i = 0 THEN  $\langle \rangle$ 
ELSE IF Test(s[i]) THEN Append(F[i - 1], s[i])
ELSE F[i - 1]

IN F[Len(s)]

```

```

MODULE FiniteSets
(*****
(* The two definitions in this standard module are overridden by TLC in *)
(* the Java class tlc2.module.FiniteSets. Each operator is overridden by *)
(* the Java method with the same name. *)
(***** )

LOCAL INSTANCE Naturals
LOCAL INSTANCE Sequences
(*****
(* Imports the definitions from Naturals and Sequences, but doesn't *)
(* export them. *)
(***** )

IsFiniteSet(S)  $\triangleq$ 
(*****
(* A set S is finite iff there is a finite sequence containing all its *)
(* elements. *)
(***** )

 $\exists seq \in Seq(S) : \forall s \in S : \exists n \in 1 \dots Len(seq) : seq[n] = s$ 

Cardinality(S)  $\triangleq$ 

```

```

(*****
(* Cardinality is defined only for finite sets. *)
(*****
LET  $CS[T \in \text{SUBSET } S] \triangleq$  IF  $T = \{\}$  THEN 0
      ELSE  $1 + CS[T \setminus \{\text{CHOOSE } x : x \in T\}]$ 
IN  $CS[S]$ 

```

```

----- MODULE Bags -----
(*****
(* A bag, also called a multiset, is a set that can contain multiple *)
(* copies of the same element. A bag can have infinitely many elements, *)
(* but only finitely many copies of any single element. *)
(* *)
(* We represent a bag in the usual way as a function whose range is a *)
(* subset of the positive integers. An element  $e$  belongs to bag  $B$  iff  $e \in B$  *)
(* is in the domain of  $B$ , in which case bag  $B$  contains  $B[e]$  copies of  $e$ . *)
(*****
(*****
(* All these definitions other than SubBag are overridden by TLC in the *)
(* Java class tlc2.module.Bags. Each operator is overridden by the Java *)
(* method with the same name, except that the mapping for TLA+ infix *)
(* operators is defined in the static block at the beginning of the Java *)
(* class. *)
(*****
EXTENDS TLC
LOCAL INSTANCE Naturals

 $IsABag(B) \triangleq$ 
  (*****
  (* True iff  $B$  is a bag. *)
  (*****
  )  $B \in$ 
  [DOMAIN  $B \rightarrow \{n \in \text{Nat} : n > 0\}]$ 

 $BagToSet(B) \triangleq$  DOMAIN  $B$ 
  (*****
  (* The set of elements at least one copy of which is in  $B$ . *)
  (*****

 $SetToBag(S) \triangleq$  [ $e \in S \mapsto 1$ ]
  (*****
  (* The bag that contains one copy of every element of the set  $S$ . *)
  (*****

 $BagIn(e, B) \triangleq$   $e \in BagToSet(B)$ 
  (*****
  (* The  $\in$  operator for bags. *)
  (*****

 $EmptyBag \triangleq$   $SetToBag(\{\})$ 

 $B1 \oplus B2 \triangleq$ 
  (*****
  (* The union of bags  $B1$  and  $B2$ . *)

```

```

(***** ) [e ∈
(DOMAIN B1) ∪ (DOMAIN B2) ↦
  (IF e ∈ DOMAIN B1 THEN B1[e] ELSE 0)
  + (IF e ∈ DOMAIN B2 THEN B2[e] ELSE 0) ]
B1 ⊖ B2 ≜
(***** )
(* The bag B1 with the elements of B2 removed—that is, with one copy *)
(* of an element removed from B1 for each copy of the same element in *)
(* B2. If B2 has at least as many copies of e as B1, then B1 ⊖ B2 *)
(* has no copies of e. *)
(***** ) LET B ≜
[e ∈ DOMAIN B1 ↦ IF e ∈ DOMAIN B2 THEN B1[e] − B2[e]
  ELSE B1[e]]
IN [e ∈ {d ∈ DOMAIN B : B[d] > 0} ↦ B[e]]
LOCAL Sum(f) ≜
(***** )
(* The sum of f[x] for all x in DOMAIN f. The definition assumes *)
(* that f is a Nat-valued function and that f[x] equals 0 for all *)
(* but a finite number of elements x in DOMAIN f. *)
(***** )
LET DSum[S ∈ SUBSET DOMAIN f] ≜
  LET elt ≜ CHOOSE e ∈ S : TRUE
  IN IF S = {} THEN 0
  ELSE f[elt] + DSum[S \ {elt}]
IN DSum[DOMAIN f]
BagUnion(S) ≜
(***** )
(* The bag union of all elements of the set S of bags. *)
(***** )
[e ∈ UNION {BagToSet(B) : B ∈ S} ↦
  Sum([B ∈ S ↦ IF BagIn(e, B) THEN B[e] ELSE 0]) ]
B1 ⊆ B2 ≜
(***** )
(* The subset operator for bags. B1 ⊆ B2 iff, for all e, bag *)
(* B2 has at least as many copies of e as bag B1 does. *)
(***** )
∧ (DOMAIN B1) ⊆ (DOMAIN B2)
∧ ∀ e ∈ DOMAIN B1 : B1[e] ≤ B2[e]
SubBag(B) ≜
(***** )
(* The set of all subbags of bag B. *)
(* *)
(* The following definition is not the one described in the TLA+ book, *)
(* but rather one that TLC can evaluate. *)
(***** )
LET RemoveFromDom(x, f) ≜ [y ∈ (DOMAIN f) \ {x} ↦ f[y]]
Combine(x, BagSet) ≜

```

```

    BagSet  $\cup$ 
    {[y  $\in$  (DOMAIN f)  $\cup$  {x}]  $\mapsto$  IF y = x THEN i ELSE f[y]} :
    f  $\in$  BagSet, i  $\in$  1 .. B[x]}
    Biggest  $\triangleq$  LET Range  $\triangleq$  {B[x] : x  $\in$  DOMAIN B} IN IF Range = {} THEN 0
    ELSE CHOOSE r  $\in$  Range :  $\forall s \in$  Range : r  $\geq$  s
    RSB[BB  $\in$  UNION {[S  $\rightarrow$  1 .. Biggest] : S  $\in$  SUBSET DOMAIN B}]  $\triangleq$ 
    IF BB =  $\langle \rangle$  THEN {}
    ELSE LET x  $\triangleq$  CHOOSE x  $\in$  DOMAIN BB : TRUE
    IN Combine(x, RSB[RemoveFromDom(x, BB)])
IN RSB[B]

(***** Here is the definition from the TLA+ book. *****)
LET AllBagsOfSubset  $\triangleq$ 
(***** )
(* The set of all bags SB such that BagToSet(SB)  $\subseteq$  *)
(* BagToSet(B). *)
(***** )
UNION {[SB  $\rightarrow$  {n  $\in$  Nat : n > 0}] : SB  $\in$  SUBSET BagToSet(B)}
IN {SB  $\in$  AllBagsOfSubset :  $\forall e \in$  DOMAIN SB : SB[e]  $\leq$  B[e]}
(***** )

BagOfAll(F(-), B)  $\triangleq$ 
(***** )
(* The bag analog of the set {F(x) : x  $\in$  B} for a set B. It's the bag *)
(* that contains, for each element e of B, one copy of F(e) for every *)
(* copy of e in B. This defines a bag iff, for any value v, the set of *)
(* e in B such that F(e) = v is finite. *)
(***** )
[e  $\in$  {F(d) : d  $\in$  BagToSet(B)}  $\mapsto$ 
Sum([d  $\in$  BagToSet(B)  $\mapsto$  IF F(d) = e THEN B[d] ELSE 0])]

BagCardinality(B)  $\triangleq$ 
(***** )
(* If B is a finite bag (one such that BagToSet(B) is a finite set), *)
(* then this is its cardinality (the total number of copies of elements *)
(* in B). Its value is unspecified if B is infinite. *)
(***** )
Sum(B)

CopiesIn(e, B)  $\triangleq$ 
(***** )
(* If B is a bag, then CopiesIn(e, B) is the number of copies of e in *)
(* B. If  $\sim$ BagIn(e, B), then CopiesIn(e, B) = 0. *)
(***** )
IF BagIn(e, B) THEN B[e] ELSE 0

```

(* Last modified on Fri 26 Jan 2007 at 8:45:03 PST by lamport *)

6 Apr 99 : Modified version for standard module set

7 Dec 98 : Corrected error found by Stephan Merz.

6 Dec 98 : Modified comments based on suggestions by Lyle Ramshaw.

5 Dec 98 : Initial version.

```

MODULE  TLC
(***** )
(* This is a standard module for use with the TLC model checker. *)
(* Operators not explained by comments here are explained in the book *)
(* "Specifying Systems". *)
(* *)
(* The definitions of all the operators in this module are overridden by *)
(* TLC with methods defined in the Java class tlc2.module.TLC. Each *)
(* definition is overridden with the method of the same name, except that *)
(* the mapping from infix operators to Java methods is specified in the *)
(* static block at the beginning of the Java class. *)
(***** )
LOCAL INSTANCE  Naturals
LOCAL INSTANCE  Sequences
LOCAL INSTANCE  FiniteSets

```

```

Print(out, val)  $\triangleq$  val
PrintT(out)  $\triangleq$  TRUE
(***** )
(* This expression equals TRUE, but evaluating it causes TLC to print *)
(* the value of out. *)
(***** )

Assert(val, out)  $\triangleq$  IF val = TRUE THEN TRUE
                  ELSE CHOOSE v : TRUE
JavaTime  $\triangleq$  CHOOSE n : n  $\in$  Nat
(***** )
(* TLC can read and set a special list of values while evaluating *)
(* expressions using the operators TLCSet and TLCGet. When TLC evaluates *)
(* TLCSet(i, v), for any positive integer i and arbitrary value v, it *)
(* obtains the value TRUE and sets element number i of the list to v. *)
(* When TLC evaluates TLCGet(i), the value it obtains is the current value *)
(* of the element number i of this list. *)
(* *)
(* One use of this feature is to check TLC's progress during long *)
(* computations. For example, suppose TLC is evaluating a formula *)
(*  $\forall x \in S : P$  where S is a large set, so it evaluates P many times. *)
(* You can use TLCGet, TLCSet, and Print to print something after every *)
(* 1000 times TLC evaluates P. *)
(* *)
(* As explained in the description of the TLCEval operator below, you may *)
(* also want to use this feature to count how many times TLC is evaluating *)
(* an expression e. To use value number i as the counter, just replace e *)
(* by *)
(* *)
(* IF TLCSet(i, TLCGet(i) + 1) THEN e ELSE 42 *)
(* *)
(* (The ELSE expression is never evaluated.) *)
(* *)
(* TLCGet accepts some pre-defined string values to query TLC state. The *)

```

```

(* values are as follows : *)
( * *)
( * – TLCGet(“distinct”) evaluates to the total number of distinct states *)
( * found by TLC so far, globally. *)
( * – TLCGet(“queue”) evaluates to the number of states currently in the *)
( * queue to be checked. *)
( * – TLCGet(“duration”) evaluates to the number of seconds elapsed since *)
( * model checking began. *)
( * – TLCGet(“diameter”) evaluates to the length of the longest behavior *)
( * found by TLC so far, globally (equals one in the initial predicate).*)
( * – TLCGet(“level”) is the length of the current behavior (equals zero *)
( * in the evaluation of the initial predicate). *)
( * *)
(* For reasons of efficiency, TLCGet and TLCSet behave somewhat strangely *)
(* when TLC is run with multiple worker threads. Each worker thread *)
(* maintains its own individual copy of the list of values on which it *)
(* evaluates TLCGet and TLCSet. The worker threads are activated only *)
(* after the computation and invariance checking of the initial states. *)
(* Before then, evaluating TLCSet(i, v) sets the element i of the list *)
(* maintained by all threads. Thus, the lists of all the worker threads *)
(* can be initialized by putting the appropriate TLCSet expression in an *)
( * ASSUME expression or in the initial predicate. *)
(***** )
TLCGet(i)  $\triangleq$  CHOOSE n : TRUE
TLCSet(i, v)  $\triangleq$  TRUE



---


d:> e  $\triangleq$  [x  $\in$  {d}  $\mapsto$  e]
f @@ g  $\triangleq$  [x  $\in$  (DOMAIN f)  $\cup$  (DOMAIN g)  $\mapsto$  IF x  $\in$  DOMAIN f THEN f[x] ELSE g[x]]
Permutations(S)  $\triangleq$ 
{f  $\in$  [S  $\rightarrow$  S] :  $\forall w \in S : \exists v \in S : f[v] = w$ }



---


(***** )
(* In the following definition, we use Op as the formal parameter rather *)
(* than < because TLC Version 1 can’t handle infix formal parameters. *)
(***** )
SortSeq(s, Op(–, –))  $\triangleq$ 
LET Perm  $\triangleq$  CHOOSE p  $\in$  Permutations(1 .. Len(s)) :  $\forall i, j \in 1 .. Len(s) :$ 
(i < j)  $\Rightarrow$  Op(s[p[i]], s[p[j]])  $\vee$  (s[p[i]] = s[p[j]])
IN [i  $\in$  1 .. Len(s)  $\mapsto$  s[Perm[i]]]

(***** )
(* TLC evaluates RandomElement(S) to be a pseudo-randomly chosen element *)
(* of the set S, where each element of S is chosen with equal probability. *)
(* This feature was added to enable the computation of statistical *)
(* properties of a specification’s executions by running TLC in simulation *)
(* mode. We don’t know if anyone has ever done this. *)
( * *)
(* In breadth-first search model checking, the pseudo-random choices made *)
(* when computing possible steps satisfying the next-state relation are *)
(* determined by the first state of the step. Thus, the choices made for *)

```

```

(* a particular state will be the same in successive runs of TLC. This is *)
(* done to permit TLC to generate an error trace if an error is found. *)
(* This applies only when TLC is run in breadth-first search mode. The *)
(* random choices made in simulation mode are independent of the state for *)
(* which they are made. *)
(***** )
RandomElement(s)  $\triangleq$  CHOOSE x  $\in$  s : TRUE

(***** )
(* The constant value Any has the special property that, for any value v, *)
(* TLC evaluates the expression v  $\in$  Any to equal TRUE. The special *)
(* value Any was introduced because TLA+ originally allowed only functions *)
(* to be defined recursively, and it was sometimes convenient to define a *)
(* function with domain Any. It is retained for backwards compatibility. *)
(***** ) Any  $\triangleq$ 
CHOOSE x : TRUE

ToString(v)  $\triangleq$  (CHOOSE x  $\in$  [a : v, b : STRING] : TRUE).b
(***** )
(* This equals a string that is the TLA+ representation of the value *)
(* that TLC obtains by evaluating v. *)
(***** )

(***** )
(* TLC often uses lazy evaluation. For example, it may not enumerate the *)
(* elements of a set of the form  $\{x \in T : P(x)\}$  unless it has to; and it *)
(* doesn't have to if it only needs to check if an element e is in that *)
(* set. (TLC can do that by evaluating  $x \in T$  and  $P(e)$ .) TLC uses *)
(* heuristics to determine when it should completely evaluate an *)
(* expression. Those heuristics work well most of the time. However, *)
(* sometimes lazy evaluation can result in the expression ultimately being *)
(* evaluated multiple times instead of just once. This can especially be *)
(* a problem when evaluating a recursively defined operator. You can get *)
(* TLC to fully evaluate an expression exp and not use lazy evaluation by *)
(* replacing exp with TLCEval(exp). *)
(* *)
(* If TLC is taking a long time to evaluate something, you can check if *)
(* lazy evaluation is the source of the problem by using the TLCSet and *)
(* TLCGet operators to count how many times expressions are being *)
(* evaluated. *)
(***** )
TLCEval(v)  $\triangleq$  v

```

— MODULE *TLCExt* —

LOCAL INSTANCE *TLC*

LOCAL INSTANCE Integers

```

(***** )
(* Imports the definitions from the modules, but doesn't export them. *)
(***** )

```

\ * *AssertEq*(*a*, *b*) is logically equivalent to the expression '*a* = *b*'. If *a* and *b* are not equal,
 \ * however, *AssertEq*(*a*, *b*) will print out the values of '*a*' and '*b*'. If the two values are equal,
 \ * nothing will be printed.

AssertEq(*a*, *b*) $\triangleq$
 IF *a* # *b* THEN
 $\wedge$ *PrintT*("Assertion failed")
 $\wedge$ *PrintT*(*a*)
 $\wedge$ *PrintT*(*b*)
 $\wedge$ *a* = *b*
 ELSE *a* = *b*

AssertError(*err*, *exp*) $\triangleq$
 LET *FailsEval*(*e*) $\triangleq$ CHOOSE *b* $\in$ BOOLEAN : TRUE \ * Expression failed to evaluate.
 TLCErr $\triangleq$ CHOOSE *s* $\in$ STRING : TRUE \ * *TLCErr* error string.
 IN IF *FailsEval*(*exp*) THEN *Assert*(*err* = *TLCErr*, *TLCErr*) ELSE TRUE

TLCGetOrDefault(*key*, *defaultVal*) $\triangleq$
 CHOOSE *n* : TRUE

TLCGetAndSet(*key*, *Op*(-, -), *val*, *defaultVal*) $\triangleq$
 LET *oldVal* $\triangleq$ *TLCGetOrDefault*(*key*, *defaultVal*)
 IN CHOOSE *v* $\in$ {*oldVal*} : *TLCSet*(*key*, *Op*(*oldVal*, *val*))

(* *HERE* BE DRAGONS! The operators below are experimental! You will probably not need them! *)

Trace $\triangleq$
 (*****)
 (* The sequence of states (represented as a record whose DOMAIN is the set of *)
 (* a spec's variables) from an initial state to the current state. In other *)
 (* words, a prefix - of a behavior - ending with the current state. A way to *)
 (* think about this operator is to see it as an implicit history variable *)
 (* that does not have to be explicitly specified at the level of the spec. *)
 (* *)
 (* Note that *op* is incompatible with *TLC!RandomElement* and Randomization (see *)
 (* *tlc2.tool.TLCTrace.getTrace(LongVec)*) and will cause *TLC* to crash. This *)
 (* technical limitation could be removed though. *)
 (* *)
 (* Beware that *Trace* is prohibitively expensive when evaluated as part of *)
 (* the next-state relation, invariants, properties, or any constraint in *)
 (* model-checking mode. Consider using a real history variable instead. *)
 (*****)
 TRUE \ * *TODO*

CounterExample $\triangleq$
 (*****)
 (* In the scope of POSTCONDITION, the operator *CounterExample* is equivalent *)
 (* to a (directed) graph represented as [state: States, action: Actions] *)

```

(* with
(* States  $\subseteq$  [ Variables  $\rightarrow$  Values ]
(* and
(* Actions  $\subseteq$ 
(* (CounterExample.state  $\times$  Action  $\times$  CounterExample.state).
(* If TLC found no violations, then CounterExample.states equals {} and
(* CounterExample.actions equals {}. If only initial state violates a
(* (safety) property, CounterExample.state is a set of those states, and
(* CounterExample.actions = {}.
(***** )
TRUE

ToTrace(CE)  $\triangleq$ 
(* This definition is implemented by the TLCExt#toTrace Java module override. It is
commented because FiniteSetsExt is part of the CommunityModules, but not tla2tools.

LET F  $\triangleq$  INSTANCE FiniteSetsExt
IN F!FoldSet(LAMBDA a, acc : acc @@ [n  $\in$  {a[1]}  $\mapsto$  a[2]], [n  $\in$  {}  $\mapsto$ 
n], CounterExample.state)

*)
TRUE

```

```

TLCModelValue(str)  $\triangleq$ 
(***** )
(* A model value is an unspecified value that TLC considers to be unequal to *)
(* any value that you can express in TLA+. *)
(*
(* In most cases, TLC model values can and should be declared via the TLC
(* config file. However, for large sets of model values, it can become
(* tedious to enumerate them explicitly. Large sets of model values usually
(* appear when simulating TLA+ specs with TLC. The following example defines *)
(* a set of 32 model values :
(*
(* SetOfModelValues  $\triangleq$  { TLCModelValue("T_MV"  $\circ$  ToString(i)) : i  $\in$  1 .. 32 } *)
(*
(* As a matter of fact, the example defines a set of "typed" model values : *)
(* TLC considers a typed model value to be unequal to any other model value *)
(* of the same type. However, it produces an error when evaluating an *)
(* expression requires it to determine if a typed model value is equal to any *)
(* value other than a model value of the same type or an ordinary model *)
(* value. *)
(*
(* A model-value type consists of a single letter, so there are 52 different *)
(* types because a and A are different types. (TLC actually accepts *)
(* digits and underscore as types; don't use them.) A model value has type *)
(* T if and only if its name begins with the two characters T_ . *)
(*
(* TLCModelValue may only appear in constant definitions! Expect bogus *)
(* behavior if TLCModelValue appears in the behavior spec, constraints, *)

```

```

(* invariants, or properties. *)
(***** )
CHOOSE  $v$ :  $ToString(v) = str$ 

 $TLCDefer(expression) \triangleq$ 
(***** )
(* Defer evaluation of expression to some later time or never. *)
(* *)
(* For  $TLC$ 's simulation mode, later is defined as the point of time when *)
(* simulation has chosen a successor state to extend the prefix of the *)
(* current behavior. *)
(* *)
(* A use case is  $TLCDefer(TLCSet(42, someVal))$ , which sets  $someVal$  only *)
(* for the states of the behavior and not the set of all successor states *)
(* of all states in the behavior. *)
(***** )
TRUE



---



 $TLCNoOp(val) \triangleq$ 
(***** )
(* No-operation operator (does nothing). Only useful to debug  $TLC$ 's *)
(* evaluator that is written in Java: Insert  $TLCNoOp$  into the expression *)
(* whose evaluation you wish to debug and set a breakpoint in this *)
(* operator's Java module override  $TLCExt\#tlcNoOp$  in  $TLCExt.java$ . *)
(***** )
 $val$ 

 $PickSuccessor(exp) \triangleq$ 
(***** )
(* When set as an action constraint in the configuration file, interactively *)
(* pick successor states during state exploration, iff the expression  $exp$  *)
(* evaluates to FALSE. To always pick successor states manually, use *)
(*  $PickSuccessor(FALSE)$ . To pick successor states when the current prefix of *)
(* behaviors exceeds 22 states, use  $PickSuccessor(TLCGet("level") < 23)$ . *)
(* Evaluates to TRUE when evaluated in the context of a constant- or *)
(* state-level formula. Also evaluates to TRUE if  $PickSuccessor$  appears as *)
(* part of the next-state relation and the successor state has not been *)
(* fully determined yet. *)
(***** ) IF
( $exp$ )
THEN TRUE
ELSE CHOOSE  $bool \in \text{BOOLEAN}$  : TRUE



---



 $TLCCache(expression, closure) \triangleq$ 
(***** )
(* Equals the value that an earlier evaluation of the expression evaluated to *)
(* iff the closure of the earlier evaluation and this evaluation are equal. *)
(* *)
(* Works in constant and state-level contexts. *)

```

```

(***** ) ex-
pression

$$TLCFP(var) \triangleq$$

(***** )
(* Equals the value that TLC gives to the given input when the value is *)
(* being fingerprinted. *)
(* Implementation note: Equals the lower 32 bits until TLC supports longs *)
(***** )
CHOOSE  $i \in Int$ : TRUE



---



$$TLCEvalDefinition(definitionName) \triangleq$$

(***** )
(* Equals the value of the definition with the given name. *)
(* Throws an error if the definition cannot be found. *)
(***** )
TRUE \ * TODO



---


MODULE Randomization
(***** )
(* This module defines operators for choosing pseudo-random subsets of  $a$  *)
(* set. It is useful for inductive invariance checking, where the *)
(* operators appear only in the initial predicate. However, it may have *)
(* other uses. *)
(* *)
(* In breadth-first search model checking, the pseudo-random choices made *)
(* when computing possible steps satisfying the next-state relation are *)
(* determined by the first state of the step. Thus, the choices made for *)
(*  $a$  particular state will be the same in successive runs of TLC. This is *)
(* done to permit TLC to generate an error trace if an error is found. *)
(* This applies only when TLC is run in breadth-first search mode. In *)
(* particular, the choices made in simulation mode are independent of the *)
(* state for which they are made. *)
(***** )
(***** )
(* Except for TestRandomSetOfSubsets, all these definitions are overridden *)
(* by TLC in the Java class tlc2.module.Randomization. Each operator is *)
(* overridden by the Java method with the same name. *)
(***** )
LOCAL INSTANCE Naturals
LOCAL INSTANCE FiniteSets
(***** )
(* RandomSubset( $k, S$ ) equals a randomly chosen subset of  $S$  containing  $k$  *)
(* elements, where  $0 < k < Cardinality(S)$ . *)
(***** )

$$RandomSubset(k, S) \triangleq \text{CHOOSE } T \in \text{SUBSET } S : Cardinality(T) = k$$

(***** )

```

```

(* RandomSetOfSubsets( $k, n, S$ ) equals a pseudo-randomly chosen set of *)
(* subsets of  $S$  – that is, a randomly chosen subset of SUBSET  $S$ . Thus, *)
(* each element  $T$  of this set is a subset of  $S$ . Each such  $T$  is chosen so *)
(* that each element of  $S$  has a probability  $n / \text{Cardinality}(S)$  of being in *)
(*  $T$ . Thus, the average number of elements in each chosen subset  $T$  is  $n$ . *)
(* The set RandomSetOfSubsets( $k, n, S$ ) is obtained by making  $k$  such *)
(* choices of  $T$ . Because this can produce duplicate choices, the number *)
(* of elements  $T$  in this set may be less than  $k$ . The average number of *)
(* elements in RandomSetOfSubsets( $k, n, S$ ) seems to be difficult to *)
(* compute in general. However, there is very little variation in the *)
(* actual number of elements in the chosen set for fixed values of  $k, n$ , *)
(* and Cardinality( $S$ ). You can therefore use the operator *)
(* TestRandomSetOfSubsets defined below to find out approximately how *)
(* close to  $k$  the cardinality of the chosen set of subsets is. *)
(***** )
RandomSetOfSubsets( $k, n, S$ )  $\triangleq$  CHOOSE  $T \in$  SUBSET SUBSET  $S$  :
  Cardinality( $T$ )  $\leq k$ 

(***** )
(* The value of TestRandomSetOfSubsets( $k, n, S$ ) is a sequence of five *)
(* values that are the cardinality of the set of subsets produced by five *)
(* executions of RandomSetOfSubsets( $k, n, S$ ). For constant values of  $k$ , *)
(*  $n$ , and  $S$ , you can enter TestRandomSetOfSubsets( $k, n, S$ ) in the Evaluate *)
(* Constant Expression section of a TLC model in the TLA+ Toolbox. *)
(* Running TLC will then tell you the approximate number of elements in *)
(* the set of subsets produced by RandomSetOfSubsets for these parameters. *)
(* You can then choose  $k$  to obtain a set of the desired size. *)
(***** )
TestRandomSetOfSubsets( $k, n, S$ )  $\triangleq$ 
   $[i \in 1..5 \mapsto \text{Cardinality}(\text{RandomSetOfSubsets}(k, n, S))]$ 

```

```

----- MODULE RealTime -----
(***** )
(* This standard module is described in Chapter 9 of the book “Specifying *)
(* Systems”. The type of specification described there cannot be handled *)
(* by the TLC model checker. *)
(***** )
EXTENDS Reals VARIABLE now

RTBound( $A, v, D, E$ )  $\triangleq$ 
  LET TNext( $t$ )  $\triangleq t' = \text{IF } \langle A \rangle_v \vee \neg(\text{ENABLED } \langle A \rangle_v)$ 
    THEN 0
    ELSE  $t + (\text{now}' - \text{now})$ 

  Timer( $t$ )  $\triangleq (t = 0) \wedge \Box[\text{TNext}(t)]_-(t, v, \text{now})$ 

  MaxTime( $t$ )  $\triangleq \Box(t \leq E)$ 

  MinTime( $t$ )  $\triangleq \Box[A \Rightarrow t \geq D]_v$ 
IN  $\exists t : \text{Timer}(t) \wedge \text{MaxTime}(t) \wedge \text{MinTime}(t)$ 

```

```

RTnow( $v$ )  $\triangleq$  LET NowNext  $\triangleq \wedge \text{now}' \in \{r \in \text{Real} : r > \text{now}\}$ 

```

```

       $\wedge$  UNCHANGED  $v$ 
    IN    $\wedge$   $now \in Real$ 
       $\wedge \Box[NowNext]_{-now}$ 
       $\wedge \forall r \in Real : WF_{-now}(NowNext \wedge (now' > r))$ 

```

```

      MODULE Json

```

```

LOCAL INSTANCE Sequences
LOCAL INSTANCE FiniteSets
LOCAL INSTANCE TL
  (*****
   (* Imports the definitions from the modules, but doesn't export them. *)
   *****)

```

```

LOCAL IsSequence( $F, D$ )  $\triangleq$  TRUE  $\vee$  FALSE

LOCAL ToJsonValue( $F, d$ )  $\triangleq$ 
  ToString(ToString( $d$ ))  $\circ$  “:”  $\circ$  ToString( $F[d]$ )

RECURSIVE ToJsonObjectString( $-, -$ )
LOCAL ToJsonObjectString( $F, D$ )  $\triangleq$  \ * LOCAL just a hint for humans.
  LET  $d \triangleq$  CHOOSE  $d \in D$  : TRUE
  IN   IF  $D = \text{DOMAIN } F$ 
    THEN “{”  $\circ$  ToJsonValue( $F, d$ )  $\circ$  IF  $D \setminus \{d\} = \{\}$ 
      THEN “}”
      ELSE ToJsonObjectString( $F, D \setminus \{d\}$ )
    ELSE “,”  $\circ$  ToJsonValue( $F, d$ )  $\circ$  IF  $D \setminus \{d\} = \{\}$ 
      THEN “}”
      ELSE ToJsonObjectString( $F, D \setminus \{d\}$ )

RECURSIVE ToJSONArrayString( $-, -$ )
LOCAL ToJSONArrayString( $F, D$ )  $\triangleq$  \ * LOCAL just a hint for humans.
  LET  $d \triangleq$  CHOOSE  $d \in D$  : TRUE
  IN   IF  $D = \text{DOMAIN } F$ 
    THEN “[”  $\circ$  ToString( $F[d]$ )  $\circ$  IF  $D \setminus \{d\} = \{\}$ 
      THEN “]”
      ELSE ToJSONArrayString( $F, D \setminus \{d\}$ )
    ELSE “,”  $\circ$  ToString( $F[d]$ )  $\circ$  IF  $D \setminus \{d\} = \{\}$ 
      THEN “]”
      ELSE ToJSONArrayString( $F, D \setminus \{d\}$ )

RECURSIVE ToJsonString( $-, -$ )
LOCAL ToJsonString( $F, D$ )  $\triangleq$  \ * LOCAL just a hint for humans.
  IF IsFiniteSet( $F$ )  $\vee$  IsSequence( $F, D$ ) THEN
    ToJSONArrayString( $F, D$ )
  ELSE
    ToJsonObjectString( $F, D$ )

ToJson( $value$ )  $\triangleq$ 
  (*****
   (* Converts the given value to a JSON string. Records are converted to *)
   (* JSON objects, tuples to JSON arrays, and scalar values as their JSON *)
   *****)

```

```

(* representation. *)
(***** ) IF
DOMAIN value = {} THEN
  "{}"
ELSE
  toJsonString(value, DOMAIN value)
toJsonArray(value)  $\triangleq$ 
(***** )
(* Converts the given tuple value to a JSON array. *)
(***** ) IF
DOMAIN value = {} THEN
  "[]"
ELSE
  toJsonArrayString(value, DOMAIN value)
toJsonObject(value)  $\triangleq$ 
(***** )
(* Converts the given tuple value to a JSON object. *)
(***** ) IF
DOMAIN value = {} THEN
  "{}"
ELSE
  toJsonObjectString(value, DOMAIN value)
toJsonSerialize(absoluteFilename, value)  $\triangleq$ 
(***** )
(* Serializes a tuple of values to the given file as (plain) JSON. *)
(* Records will be serialized as a JSON objects, and tuples as arrays. *)
(***** )
TRUE
toJsonDeserialize(absoluteFilename)  $\triangleq$ 
(***** )
(* Deserializes JSON values from the given file. JSON objects will be *)
(* deserialized to records, and arrays will be deserialized to tuples. *)
(***** )
CHOOSE val : TRUE
ndtoJsonSerialize(absoluteFilename, value)  $\triangleq$ 
(***** )
(* Serializes a tuple of values to the given file as newline delimited *)
(* JSON. Records will be serialized as a JSON objects, and tuples as *)
(* arrays. *)
(***** )
TRUE
ndtoJsonDeserialize(absoluteFilename)  $\triangleq$ 
(***** )
(* Deserializes JSON values from the given file. JSON values must be *)
(* separated by newlines. JSON objects will be deserialized to records, *)
(* and arrays will be deserialized to tuples. *)
(***** )

```

CHOOSE *val* : TRUE

MODULE *Toolbox*

```
(*****
( *
(*****
LOCAL INSTANCE Naturals \ * a + b
LOCAL INSTANCE FiniteSets \ * Cardinality
LOCAL INSTANCE TLC \ * TLCGet
```

```
(* Trace Exploration *)
_TETrace  $\triangleq$ 
  CHOOSE f ∈ [ (Nat \ {0}) → STRING ] : TRUE
LOCAL _TETraceLength  $\triangleq$  Cardinality(DOMAIN _TETrace)

\ * When evaluated as part of a TLC ALIAS expression, _TEPosition should be
\ * redefined as TLCGet("level"), or replace with TLCGet(level) right away.
\ * The definition below accounts for several "peculiarities" of the
\ * Toolbox s trace exploration.
_TEPosition  $\triangleq$ 
  IF TLCGet("level") ≥ _TETraceLength
  THEN _TETraceLength
  ELSE IF TLCGet("level") = 0 THEN 1 ELSE TLCGet("level")
```

MODULE *Folds*

```
(*****
(* This module provides a general framework for folding operations over *)
(* sets with flexible element selection and transformation. *)
( *
(* The main operator MapThenFoldSet is the foundation for many other *)
(* folding operations found in : *)
( * – FiniteSetsExt.FoldSet (simplified for sets) *)
( * – Functions.FoldFunction (specialized for functions) *)
( * – SequencesExt.FoldSeq (specialized for sequences) *)
( *
(*****

MapThenFoldSet(op(-, -), base, f(-), choose(-), S)  $\triangleq$ 
(*****
(* Starting from base, apply op to f(x), for all x ∈ S, by choosing the set *)
(* elements with choose If there are multiple ways for choosing an element,*)
```

```

(* op should be associative and commutative. Otherwise, the result may depend *)
(* on the concrete implementation of 'choose'. *)
(* *)
(* FoldSet, a simpler version for sets is contained in FiniteSetsEx. *)
(* FoldFunction, a simpler version for functions is contained in Functions. *)
(* FoldSequence, a simpler version for sequences is contained in SequencesExt. *)
(* *)
(* Example: *)
(* *)
(* MapThenFoldSet(LAMBDA x,y: x \cup y, *)
(*      {}, *)
(*      LAMBDA x: {{x}}, *)
(*      LAMBDA set: CHOOSE x \in set: TRUE, *)
(*      {1,2}) *)
(* *)
(*      = {{1},{2}} *)
(* *)
(*****
  LET iter[s \in SUBSET S] ==
    IF s = {} THEN base
    ELSE LET x == choose(s)
          IN op(f(x), iter[s \ {x}])
  IN iter[S]

=====

----- MODULE Functions -----
(*****
(* '\large\bf \vspace{12pt} *)
(* Notions about functions including injection, surjection, and bijection. *)
(* Originally contributed by Tom Rodeheffer, MSR. *)
(* \vspace{12pt}~' *)
(*****

EXTENDS Integers, Folds

(*****
(* Restriction of a function to a set (should be a subset of the domain). *)
(*****
Restrict(f,S) == [ x \in S |-> f[x] ]

(*****
(* Restriction of a function to the subset of its domain satisfying a *)
(* test predicate. *)
(* *)
(* Example: *)
(* (LET f == (0 :> "a" @@ 1 :> "b" @@ 2 :> "c") *)
(* IN RestrictDomain(f, LAMBDA x : x \in {0,2})) *)
(* = (0 :> "a" @@ 2 :> "c") *)
(*****
RestrictDomain(f, Test(_)) == Restrict(f, {x \in DOMAIN f : Test(x)})

```

```

(*****
(* Restriction of a function to the subset of its domain for which the
(* function values satisfy a test predicate.
(*
(* Example:
(* (LET f == ("a" :> 0 @@ "b" :> 1 @@ "c" :> 2)
(* IN RestrictValues(f, LAMBDA y : y \in {0,2}))
(* = ("a" :> 0 @@ "b" :> 2)
(*
(* This is similar to the operator SelectSeq from the standard Sequences
(* module and related to standard "filter" functions in functional
(* programming. However, SelectSeq produces sequences, whereas
(* RestrictValues will in general not. For example,
(*
(* RestrictValues([0,1,2], LAMBDA y : y \in {0,2})
(* = (1 :> 0 @@ 3 :> 2)
(*****
RestrictValues(f, Test(_)) ==
  LET S == {x \in DOMAIN f : Test(f[x])}
  IN Restrict(f, S)

(*****
(* Check if a function narrow is a restriction of a function wide, i.e.
(* Is the domain of narrow a subset of that of wide, and does the
(* projection of wide on the domain of narrow have the same image as
(* narrow does.
(*
(* Examples:
(* IsRestriction([one |-> 1], [one |-> 1, two |-> 2])
(* IsRestriction([one |-> 1], [one |-> 1])
(* ~IsRestriction([one |-> 1, two |-> 2], [one |-> 1, two |-> 3])
(* ~IsRestriction([one |-> 1], [2 |-> two])
(* ~IsRestriction([one |-> 1, two |-> 2], [two |-> 2])
(*****
IsRestriction(narrow, wide) ==
  /\ DOMAIN narrow \subseq DOMAIN wide
  /\ \A x \in DOMAIN narrow \intersect DOMAIN wide: narrow[x] = wide[x]

(*****
(* Range of a function.
(* Note: The image of a set under function f can be defined as
(* Range(Restrict(f,S)).
(*****
Range(f) == { f[x] : x \in DOMAIN f }

(*****
(* Assuming DOMAIN f \subseq DOMAIN g, apply the binary operation T to
(* the corresponding elements of the two functions f and g.
(*
(* Example:
(* LET f == ("a" :> 0 @@ "b" :> 1 @@ "c" :> 2)
(* g == ("a" :> 1 @@ "b" :> 1 @@ "c" :> 3)
(* IN Pointwise(f,g,+) = ("a" :> 1 @@ "b" :> 2 @@ "c" :> 5 )
(*****
Pointwise(f, g, T(_, _)) == [ e \in DOMAIN f |-> T(f[e], g[e]) ]

```

```

(*****
(* The inverse of a function. *)
(* Example: *)
(*   LET f == ("a" :> 0 @@ "b" :> 1 @@ "c" :> 2) *)
(*   IN Inverse(f, DOMAIN f, Range(f)) = *)
(*           (0 :> "a" @@ 1 :> "b" @@ 2 :> "c") *)
(* Example: *)
(*   LET f == ("a" :> 0 @@ "b" :> 1 @@ "c" :> 2) *)
(*   IN Inverse(f, DOMAIN f, {1,3}) = *)
(*           1 :> "b" @@ 3 :> "a") *)
(*****
Inverse(f,S,T) == [t \in T |-> CHOOSE s \in S : t \in Range(f) => f[s] = t]

(*****
(* The inverse of a function. *)
(*****
AntiFunction(f) == Inverse(f, DOMAIN f, Range(f))

(*****
(* A function is injective iff it maps each element in its domain to a *)
(* distinct element. *)
(* *)
(* This definition is overridden by TLC in the Java class Functions.java *)
(* The operator is overridden by the Java method with the same name. *)
(*****
IsInjective(f) == \A a,b \in DOMAIN f : f[a] = f[b] => a = b

(*****
(* Set of injections between two sets. *)
(*****
Injection(S,T) == { M \in [S -> T] : IsInjective(M) }

(*****
(* A map is a surjection iff for each element in the range there is some *)
(* element in the domain that maps to it. *)
(*****
Surjection(S,T) == { M \in [S -> T] : \A t \in T : \E s \in S : M[s] = t }

(*****
(* A map is a bijection iff it is both an injection and a surjection. *)
(*****
Bijection(S,T) == Injection(S,T) \cap Surjection(S,T)

(*****
(* An injection, surjection, or bijection exists if the corresponding set *)
(* is nonempty. *)
(*****
ExistsInjection(S,T) == Injection(S,T) # {}
ExistsSurjection(S,T) == Surjection(S,T) # {}
ExistsBijection(S,T) == Bijection(S,T) # {}

```

```

(*****
(* Applies the binary operator op on the elements f[i] for i \in indices, *)
(* a subset of DOMAIN f, in an unspecified order, starting from value *)
(* base. The result is *)
(* op(f[i],op(f[j], ..., op(f[k],base) ...)) *)
(* where i,j,k range over the sets indices. *)
(* *)
(* op should be associative and commutative, because we can not assume a *)
(* particular ordering of i, j, and k *)
(* *)
(* Example: *)
(* FoldFunctionOnSet(LAMBDA x,y: {x} \cup y, {}, <<1,2>>, {}) = {} *)
(*****
FoldFunctionOnSet(op(_,_), base, fun, indices) ==
  MapThenFoldSet(op, base, LAMBDA i : fun[i], LAMBDA s: CHOOSE x \in s : TRUE, indices)

(*****
(* Special case of the above where indices = DOMAIN fun. In other words, *)
(* the values f[i] for all i \in DOMAIN fun, are combined using the binary *)
(* operator op, in an unspecified order. *)
(* *)
(* Example: *)
(* FoldFunction(LAMBDA x,y: {x} \cup y, {}, <<1,2,1>>) = {1,2} *)
(*****
FoldFunction(op(_,_), base, fun) ==
  FoldFunctionOnSet(op, base, fun, DOMAIN fun)

(*****
(* Sum the values f[i] for i \in indices, a subset of DOMAIN f. *)
(* *)
(* Example: *)
(* SumFunctionOnSet(<<4,5,6>>, {1,2}) = 9 *)
(* SumFunction(<<4,5,6>>) = 15 *)
(*****
SumFunctionOnSet(fun, indices) == FoldFunctionOnSet(+, 0, fun, indices)
SumFunction(fun) == SumFunctionOnSet(fun, DOMAIN fun)

=====
\* Created Thu Apr 11 10:30:48 PDT 2013 by tomr

----- MODULE Relation -----
LOCAL INSTANCE Naturals
LOCAL INSTANCE FiniteSets \* TODO Consider moving to a more specific module.

```

```

(*****
(* This module provides some basic operations on relations, represented *)
(* as binary Boolean functions over some set S. *)
(* *)
(* Relations are represented as functions  $R: S \times S \rightarrow \text{BOOLEAN}$ , where *)
(*  $R[x,y] = \text{TRUE}$  means that  $x$  is related to  $y$ . *)
(* *)
(* The module includes: *)
(* - Basic relation properties (reflexive, symmetric, transitive, etc.) *)
(* - Ordering relations (partial orders, total orders) *)
(* - Closure operations (transitive closure) *)
(* - Connectivity analysis *)
(* *)
(* Each property has two variants: *)
(* - Direct: Takes a relation  $R$  as a Boolean function *)
(* - Under: Takes a binary operator and constructs the relation *)
(*****

(*****
(* Is the relation  $R$  reflexive over  $S$ ? *)
(* *)
(* A relation is reflexive if every element is related to itself. *)
(* *)
(* Examples: *)
(*  $\text{IsReflexive}([\langle x,y \rangle \in \{1,2\} \times \{1,2\} \mid \rightarrow x=y], \{1,2\}) = \text{TRUE}$  *)
(*  $\text{IsReflexiveUnder}(=, \{1,2,3\}) = \text{TRUE}$  *)
(*  $\text{IsReflexiveUnder}(<, \{1,2,3\}) = \text{FALSE}$  *)
(*****
IsReflexive(R, S) ==  $\forall x \in S : R[x,x]$ 

IsReflexiveUnder(op(_,_), S) ==
    IsReflexive([\langle x,y \rangle \in S \times S \mid \rightarrow \text{op}(x,y)], S)

(*****
(* Is the relation  $R$  irreflexive over set  $S$ ? *)
(* *)
(* A relation is irreflexive if no element is related to itself. *)
(* This is the opposite of reflexive. *)
(* *)
(* Examples: *)
(*  $\text{IsIrreflexiveUnder}(<, \{1,2,3\}) = \text{TRUE}$  *)
(*  $\text{IsIrreflexiveUnder}(\leq, \{1,2,3\}) = \text{FALSE}$  *)
(*****
IsIrreflexive(R, S) ==  $\forall x \in S : \sim R[x,x]$ 

IsIrreflexiveUnder(op(_,_), S) ==
    IsIrreflexive([\langle x,y \rangle \in S \times S \mid \rightarrow \text{op}(x,y)], S)

(*****
(* Is the relation  $R$  symmetric over set  $S$ ? *)
(* *)
(* A relation is symmetric if  $x$  related to  $y$  implies  $y$  related to  $x$ . *)
(* *)
(* Examples: *)
(*  $\text{IsSymmetricUnder}(=, \{1,2,3\}) = \text{TRUE}$  *)
(*  $\text{IsSymmetricUnder}(<, \{1,2,3\}) = \text{FALSE}$  *)
(*****
IsSymmetric(R, S) ==  $\forall x,y \in S : R[x,y] \Leftrightarrow R[y,x]$ 

```

```

IsSymmetricUnder(op(_,_), S) ==
  IsSymmetric([<<x,y>> \in S \X S |-> op(x,y)], S)

(*****)
(* Is the relation R antisymmetric over set S? *)
(* *)
(* A relation is antisymmetric if x related to y and y related to x *)
(* implies x equals y. *)
(* *)
(* Examples: *)
(* IsAntiSymmetricUnder(<=, {1,2,3}) = TRUE *)
(* IsAntiSymmetricUnder(=, {1,2,3}) = TRUE *)
(*****)
IsAntiSymmetric(R, S) == \A x,y \in S : R[x,y] /\ R[y,x] => x=y

IsAntiSymmetricUnder(op(_,_), S) ==
  IsAntiSymmetric([<<x,y>> \in S \X S |-> op(x,y)], S)

(*****)
(* Is the relation R asymmetric over set S? *)
(* *)
(* A relation is asymmetric if it's never the case that both x is related *)
(* to y and y is related to x (unless x=y). *)
(* *)
(* Examples: *)
(* IsAsymmetricUnder(<, {1,2,3}) = TRUE *)
(* IsAsymmetricUnder(<=, {1,2,3}) = FALSE (due to reflexivity) *)
(*****)
IsAsymmetric(R, S) == \A x,y \in S : ~(R[x,y] /\ R[y,x])

IsAsymmetricUnder(op(_,_), S) ==
  IsAsymmetric([<<x,y>> \in S \X S |-> op(x,y)], S)

(*****)
(* Is the relation R transitive over set S? *)
(* *)
(* A relation is transitive if x related to y and y related to z implies *)
(* x related to z. *)
(* *)
(* Examples: *)
(* IsTransitiveUnder(<, {1,2,3}) = TRUE *)
(* IsTransitiveUnder(=, {1,2,3}) = TRUE *)
(*****)
IsTransitive(R, S) == \A x,y,z \in S : R[x,y] /\ R[y,z] => R[x,z]

IsTransitiveUnder(op(_,_), S) ==
  IsTransitive([<<x,y>> \in S \X S |-> op(x,y)], S)

```

```

(*****)
(* Is the set S strictly partially ordered under the (binary) relation R? *)
(* *)
(* A strict partial order combines irreflexivity, antisymmetry, and *)
(* transitivity. *)
(* *)
(* Examples: *)
(* IsStrictlyPartiallyOrderedUnder(<, {1,2,3}) = TRUE *)
(* IsStrictlyPartiallyOrderedUnder(<=, {1,2,3}) = FALSE (not irreflexive)*)
(*****)
IsStrictlyPartiallyOrdered(R, S) ==
  /\ IsIrreflexive(R, S)
  /\ IsAntiSymmetric(R, S)
  /\ IsTransitive(R, S)

IsStrictlyPartiallyOrderedUnder(op(_,_), S) ==
  IsStrictlyPartiallyOrdered([<x,y>> \in S \X S |-> op(x,y)], S)

(*****)
(* Is the set S (weakly) partially ordered under the (binary) relation R? *)
(* *)
(* A partial order (poset) combines reflexivity, antisymmetry, and *)
(* transitivity. *)
(* *)
(* Examples: *)
(* IsPartiallyOrderedUnder(<=, {1,2,3}) = TRUE *)
(* IsPartiallyOrderedUnder(<, {1,2,3}) = FALSE (not reflexive) *)
(*****)
IsPartiallyOrdered(R, S) ==
  /\ IsReflexive(R, S)
  /\ IsAntiSymmetric(R, S)
  /\ IsTransitive(R, S)

IsPartiallyOrderedUnder(op(_,_), S) ==
  IsPartiallyOrdered([<x,y>> \in S \X S |-> op(x,y)], S)

(*****)
(* Is the set S strictly totally ordered under the (binary) relation R? *)
(* *)
(* A strict total order is a strict partial order where any two distinct *)
(* elements are comparable (trichotomy property). *)
(* *)
(* Examples: *)
(* IsStrictlyTotallyOrderedUnder(<, {1,2,3}) = TRUE *)
(*****)
IsStrictlyTotallyOrdered(R, S) ==
  /\ IsStrictlyPartiallyOrdered(R, S)
  \* Trichotomy (R is irreflexive)
  /\ \A x,y \in S : x # y => R[x,y] \/ R[y,x]

IsStrictlyTotallyOrderedUnder(op(_,_), S) ==
  IsStrictlyTotallyOrdered([<x,y>> \in S \X S |-> op(x,y)], S)

```

```

(*****
(* Is the set S totally ordered under the (binary) relation R? *)
(* *)
(* A total order is a partial order where any two elements are comparable. *)
(* *)
(* Examples: *)
(* IsTotallyOrderedUnder(<=, {1,2,3}) = TRUE *)
(*****
IsTotallyOrdered(R, S) ==
  /\ IsPartiallyOrdered(R, S)
  /\ \A x,y \in S : R[x,y] /\ R[y,x]

IsTotallyOrderedUnder(op(_,_), S) ==
  IsTotallyOrdered([<x,y> \in S \X S |-> op(x,y)], S)

(*****
(* Compute the transitive closure of relation R over set S. *)
(*****
TransitiveClosure(R, S) ==
  LET N == Cardinality(S)
  trcl[n \in Nat] ==
    [x,y \in S |-> IF n=0 THEN R[x,y]
      ELSE /\ trcl[n-1][x,y]
            /\ \E z \in S : trcl[n-1][x,z] /\ trcl[n-1][z,y]]

  IN trcl[N]

(*****
(* Compute the reflexive transitive closure of relation R over set S. *)
(*****
ReflexiveTransitiveClosure(R, S) ==
  LET trcl == TransitiveClosure(R,S)
  IN [x,y \in S |-> x=y /\ trcl[x,y]]

(*****
(* Is the relation R connected over set S, i.e. does there exist a path *)
(* between two arbitrary elements of S? *)
(*****
IsConnected(R, S) ==
  LET rtrcl == ReflexiveTransitiveClosure(R,S)
  IN \A x,y \in S : rtrcl[x,y]

=====
\* Modification History
\* Last modified Tues Sept 17 06:20:47 CEST 2024 by Markus Alexander Kuppe
\* Created Tue Apr 26 10:24:07 CEST 2016 by merz

----- MODULE FiniteSetsExt -----
EXTENDS Integers, FiniteSets, Folds, Functions \* Replace with LOCAL INSTANCE when https://github.com/tlaplus/tla
\* LOCAL INSTANCE Integers
\* LOCAL INSTANCE FiniteSets

(*****
(* Fold op over the elements of set using base as starting value. *)
(* *)
(* Example: *)
(* FoldSet(LAMBDA x,y : x + y, 0, 0 .. 10) = 55 *)
(*****
FoldSet(op(_,_), base, set) ==
  MapThenFoldSet(op, base, LAMBDA x : x, LAMBDA s : CHOOSE x \in s : TRUE, set)

```

```

(*****)
(* Calculate the sum of the elements in set. *)
(* *)
(* Example: *)
(* SumSet(0 .. 10) = 55 *)
(*****)
SumSet(set) ==
  FoldSet(+, 0, set)

(*****)
(* Calculate the product of the elements in set. *)
(* *)
(* Example: *)
(* ProductSet(1 .. 3) = 6 *)
(*****)
ProductSet(set) ==
  FoldSet(LAMBDA x, y: x * y, 1, set)

(*****)
(* An alias for FoldSet. ReduceSet was used instead of FoldSet in *)
(* earlier versions of the community modules. *)
(*****)
ReduceSet(op(_, _), set, acc) ==
  FoldSet(op, acc, set)

(*****)
(* Calculate the sum of projections of the elements in a set. *)
(* *)
(* Example: *)
(* MapThenSumSet( *)
(* LAMBDA e : e.n, *)
(* {[n |-> 0], [n |-> 1], [n |-> 2]} *)
(* ) = 3 *)
(*****)
MapThenSumSet(op(_), set) ==
  MapThenFoldSet(+, 0, op, LAMBDA s : CHOOSE x \in s : TRUE, set)

(*****)
(* Flattens a set of sets into a single set containing all elements. *)
(* *)
(* This is equivalent to TLA+ UNION operator. *)
(* *)
(* NOTE: Use UNION directly instead of this operator. FlattenSet is only *)
(* kept for backward compatibility. *)
(* *)
(* Example: *)
(* FlattenSet({{1,2}, {2,3}, {4}}) = {1,2,3,4} *)
(* FlattenSet({}) = {} *)
(* FlattenSet({{}}) = {} *)
(*****)
FlattenSet(S) ==
  UNION S

```

```

(*****
(* The symmetric difference of two sets. *)
(* *)
(* The symmetric difference of sets A and B contains elements that are *)
(* in exactly one of A or B, but not in both. This is equivalent to *)
(*  $(A \setminus B) \cup (B \setminus A)$ . *)
(* *)
(* Examples: *)
(*  $\text{SymDiff}(\{1,2,3\}, \{2,3,4\}) = \{1,4\}$  *)
(*  $\text{SymDiff}(\{1,2\}, \{3,4\}) = \{1,2,3,4\}$  *)
(*  $\text{SymDiff}(\{1,2\}, \{1,2\}) = \{\}$  *)
(*****
SymDiff(A, B) == (A \ B) \cup (B \ A)

```

```

(*****
(* Quantify the elements in S matching the predicate (LAMBDA) P. *)
(* *)
(* This operator is overridden by FiniteSetsExt#quantify whose *)
(* implementation does *not* enumerate the intermediate set! This is *)
(* the only advantage that Quantify(...) has over Cardinality(...). *)
(* *)
(* Example: *)
(*  $\text{Quantify}(1..9, \text{LAMBDA } n : n \% 2 = 0) = 4$  *)
(*****
Quantify(S, P(_)) ==
  Cardinality({s \in S : P(s)})

```

```

(*****
(* A k-subset ks of a set S has Cardinality(ks) = k. The number of *)
(* k-subsets of a set S with Cardinality(S) = n is given by the binomial *)
(* coefficients n over k. A set S with Cardinality(S) = n has  $2^n$  *)
(* k-subsets.  $\bigcup_{k=0}^{\text{Cardinality}(S)} \text{kSubset}(k, S) = \{\}$ . *)
(* *)
(* This operator is overridden by FiniteSetsExt#kSubset whose *)
(* implementation, contrary to  $\{s \in \text{SUBSET } S : \text{Cardinality}(s) = k\}$ , *)
(* only enumerates the k-subsets of S and not all subsets. *)
(* *)
(* Example: *)
(*  $\text{kSubset}(2, 1..3) = \{\{1,2\}, \{2,3\}, \{3,1\}\}$  *)
(*****
kSubset(k, S) ==
  { s \in SUBSET S : Cardinality(s) = k }

```

```

(*****
(* We define Max(S) and Min(S) to be the maximum and minimum, *)
(* respectively, of a finite, non-empty set S of integers. *)
(*****
Max(S) == CHOOSE x \in S : \A y \in S : x >= y
Min(S) == CHOOSE x \in S : \A y \in S : x <= y

```

```

(*****
(* Compute all possible choice sets from a collection of sets. *)
(* *)
(* Given a collection of sets, this operator equals all possible sets *)
(* that can be formed by choosing exactly one element from each input set. *)
(* This is related to the Cartesian product. *)
(* *)
(* The implementation uses choice functions: functions that map each *)
(* input set to one of its elements. The range of each such function *)
(* gives us one possible choice set. *)
(* *)
(* Examples: *)
(* Choices({{1,2}, {3}}) = {{1,3}, {2,3}} *)
(* Choices({{1,2}, {2,3}, {5}}) = *)
(* {{1,2,5}, {1,3,5}, {2,3,5}, {2,5}} *)
(* (Note: {2,5} appears because we choose 2 from both sets) *)
(* Choices({}) = {{}} (empty choice from empty collection) *)
(*****)
Choices(Sets) == LET ChoiceFunction(Ts) == { f \in [Ts -> UNION Ts] :
                                         \A T \in Ts : f[T] \in T }
              IN { Range(f) : f \in ChoiceFunction(Sets) }

-----

(*****
(* Chooses unique element from the input set matching the predicate *)
(* (LAMBDA) P. *)
(* *)
(* Contrary to CHOOSE, fails with *)
(* "CHOOSE x \in S: P, but no element of S satisfied P: *)
(* not just if P(_) holds for none of the elements in S, but also if *)
(* P(_) holds for more than one element in S. *)
(* *)
(* Example: *)
(* ChooseUnique({2, 3, 4, 5}, LAMBDA x : x % 3 = 1) = 4 *)
(* *)
(*****)
ChooseUnique(S, P(_)) == CHOOSE x \in S :
                        P(x) /\ \A y \in S : P(y) => y = x

=====

----- MODULE SequencesExt -----
EXTENDS Sequences, Naturals, FiniteSets, FiniteSetsExt, Folds, Functions
\* TLAPM does not play well with LOCAL INSTANCE, reinstate the following
\* when that issue is fixed.
\* LOCAL INSTANCE Sequences
\* LOCAL INSTANCE Naturals
\* LOCAL INSTANCE FiniteSets
\* LOCAL INSTANCE FiniteSetsExt
\* LOCAL INSTANCE Folds
\* LOCAL INSTANCE Functions
LOCAL INSTANCE TLC
(*****
(* Imports the definitions from the modules, but doesn't export them. *)
(*****)

```

```

(*****
(* Convert a sequence to the set of its elements (sequence image). *)
(* *)
(* Returns the set of all elements that appear in the sequence, *)
(* effectively removing duplicates and order. The cardinality of the *)
(* result is at most the length of the sequence. *)
(* *)
(* Examples: *)
(* ToSet(<<>>) = {} *)
(* ToSet(<<1>>) = {1} *)
(* ToSet(<<1,2,3>>) = {1,2,3} *)
(* ToSet(<<1,1,2>>) = {1,2} *)
(* *)
(* See: https://en.wikipedia.org/wiki/Image\_\(mathematics\) *)
(*****
ToSet(s) ==
  { s[i] : i \in DOMAIN s }

(*****
(* Convert a set to some sequence that contains all the elements of the *)
(* set exactly once, and contains no other elements. *)
(*****
SetToSeq(S) ==
  CHOOSE f \in [1..Cardinality(S) -> S] : IsInjective(f)

(*****
(* Convert the set S to a set containing all sequences containing the *)
(* elements of S exactly once and no other elements. *)
(* Example: *)
(* SetToSeqs({}), {<<>>} *)
(* SetToSeqs({"t","l"}) = {<<"t","l">>, <<"l","t">>} *)
(*****
SetToSeqs(S) ==
  LET D == 1..Cardinality(S)
  IN { f \in [D -> S] : \A i,j \in D : i # j => f[i] # f[j] }

(*****
(* Convert a set to a sorted sequence that contains all the elements of *)
(* the set exactly once, and contains no other elements. *)
(*****
SetToSortSeq(S, op(_,_)) ==
  SortSeq(SetToSeq(S), op)

(*****
(* Convert the set S to a set containing all k-permutations of elements *)
(* of S for k \in 0..Cardinality(S). *)
(* Example: *)
(* SetToAllKPermutations({}) = {<<>>} *)
(* SetToAllKPermutations({"a"}) = {<<>>, <<"a">>} *)
(* SetToAllKPermutations({"a","b"}) = *)
(* {<<>>, <<"a">>, <<"b">>, <<"a","b">>, <<"b","a">>} *)
(*****
SetToAllKPermutations(S) ==
  UNION { SetToSeqs(s) : s \in SUBSET S }

```

```

(*****
(* TupleOf(s, 3) = s \X s \X s *)
(*****
TupleOf(set, n) ==
  [1..n -> set]

(*****
(* All sequences up to length n with all elements in set. Includes empty *)
(* sequence. *)
(*****
SeqOf(set, n) ==
  UNION {[1..m -> set] : m \in 0..n}

(*****
(* An alias for SeqOf to make the connection to Sequences!Seq, which is *)
(* the unbounded version of BoundedSeq. *)
(*****
BoundedSeq(S, n) ==
  SeqOf(S, n)

-----

(*****
(* TRUE iff the element e \in ToSet(s). *)
(*****
Contains(s, e) ==
  \E i \in 1..Len(s) : s[i] = e

(*****
(* Reverse the given sequence s: Let l be Len(s) (length of s). *)
(* Equals a sequence s.t. << S[1], S[l-1], ..., S[1]>> *)
(*****
Reverse(s) ==
  [ i \in 1..Len(s) |-> s[(Len(s) - i) + 1] ]

(*****
(* The sequence s with e removed or s iff e \notin Range(s) *)
(*****
Remove(s, e) ==
  SelectSeq(s, LAMBDA t: t # e)

(*****
(* Equals the sequence s except that all occurrences of element old are *)
(* replaced with the element new. *)
(*****
ReplaceAll(s, old, new) ==
  [i \in 1 .. Len(s) |-> IF s[i] = old THEN new ELSE s[i]]

-----

(*****
(* Selects the index of the first element such that Test(seq[i]) is true *)
(* Equals 0 if Test(seq[i]) is FALSE for all elements. *)
(*****
SelectInSeq(seq, Test(_)) ==
  LET I == { i \in 1..Len(seq) : Test(seq[i]) }
  IN IF I # {} THEN Min(I) ELSE 0

```

```

(*****
(* Selects the index of the first element in seq such that Test(seq[i]) *)
(* is TRUE for this elements in from..to. Equals 0 if Test(seq[i]) is *)
(* FALSE for all elements. *)
(*****
SelectInSubSeq(seq, from, to, Test(_)) ==
  SelectInSeq(SubSeq(seq, from, to), Test)

(*****
(* Selects the index of the last element such that Test(seq[i]) is true *)
(* Equals 0 if Test(seq[i]) is FALSE for all elements. *)
(*****
SelectLastInSeq(seq, Test(_)) ==
  LET I == { i \in 1..Len(seq) : Test(seq[i]) }
  IN IF I # {} THEN Max(I) ELSE 0

(*****
(* Selects the index of the last element in seq such that Test(seq[i]) *)
(* is TRUE for this elements in from..to. Equals 0 if Test(seq[i]) is *)
(* FALSE for all elements. *)
(*****
SelectLastInSubSeq(seq, from, to, Test(_)) ==
  SelectLastInSeq(SubSeq(seq, from, to), Test)

-----

/* The operators below up to including IsStrictSuffix have been extracted
/* from the TLAPS module SequencesTheorems.tla as of 10/14/2019. The original
/* comments have been partially rewritten.

(*****
(* Inserts element e at the position i moving the original element to i+1 *)
(* and so on. In other words, a sequence t s.t.: *)
(* /\ Len(t) = Len(s) + 1 *)
(* /\ t[i] = e *)
(* /\ \A j \in 1..(i - 1): t[j] = s[j] *)
(* /\ \A k \in (i + 1)..Len(s): t[k + 1] = s[k] *)
(*****
InsertAt(s, i, e) ==
  SubSeq(s, 1, i-1) \o <<e>> \o SubSeq(s, i, Len(s))

(*****
(* Replaces the element at position i with the element e. *)
(*****
ReplaceAt(s, i, e) ==
  [s EXCEPT ![i] = e]

(*****
(* Replaces the element at position i shortening the length of s by one. *)
(*****
RemoveAt(s, i) ==
  SubSeq(s, 1, i-1) \o SubSeq(s, i+1, Len(s))

```

```

(*****)
(* The sequence s with the first occurrence of e removed or s *)
(* iff e \notin Range(s) *)
(*****)
RemoveFirst(s, e) ==
  IF \E i \in 1..Len(s): s[i] = e
  THEN RemoveAt(s, SelectInSeq(s, LAMBDA v: v = e))
  ELSE s

(*****)
(* The sequence s with the first element removed s.t. Test(e) or s *)
(* iff e \notin Range(s) *)
(*****)
RemoveFirstMatch(s, Test(_)) ==
  IF \E i \in 1..Len(s): Test(s[i])
  THEN RemoveAt(s, SelectInSeq(s, Test))
  ELSE s
-----

(*****)
(* Cons prepends an element at the beginning of a sequence. *)
(*****)
Cons(elt, seq) ==
  <<elt>> \o seq

(*****)
(* Reverses the operands of Sequences! Append for better compatibility *)
(* with FoldFunction below. *)
(* Example: *)
(* FoldSeq(LAMBDA x,y: {x} \cup y, {}, <<1,2,1>>) = Range(<<1,2,1>>) *)
(*****)
Snoc(elt, seq) ==
  Append(seq, elt)

(*****)
(* The sequence formed by removing its last element. *)
(*****)
Front(s) ==
  SubSeq(s, 1, Len(s)-1)

(*****)
(* The last element of the sequence. *)
(*****)
Last(s) ==
  s[Len(s)]
-----

(*****)
(* TRUE iff the sequence s is a prefix of the sequence t, s.t. *)
(* \E u \in Seq(Range(t)) : t = s \o u. In other words, there exists *)
(* a suffix u that with s prepended equals t. *)
(*****)
IsPrefix(s, t) ==
  Len(s) <= Len(t) /\ SubSeq(s, 1, Len(s)) = SubSeq(t, 1, Len(s))

```

```

(*****
(* TRUE iff the sequence s is a prefix of the sequence t and s # t      *)
(*****
IsStrictPrefix(s,t) ==
  IsPrefix(s, t) /\ s # t

(*****
(* TRUE iff the sequence s is a suffix of the sequence t, s.t.          *)
(* \E u \in Seq(Range(t)) : t = u \o s. In other words, there exists a  *)
(* prefix that with s appended equals t.                                *)
(*****
IsSuffix(s, t) ==
  IsPrefix(Reverse(s), Reverse(t))

(*****
(* TRUE iff the sequence s is a suffix of the sequence t and s # t      *)
(*****
IsStrictSuffix(s, t) ==
  IsSuffix(s,t) /\ s # t

-----

(*****
(* The set of prefixes of the sequence s, including the empty sequence.  *)
(*****
Prefixes(s) ==
  \* 0.. for <<>>
  { SubSeq(s, 1, 1) : 1 \in 0..Len(s) }

(*****
(* The set of all sequences that are prefixes of the set of sequences S. *)
(*****
CommonPrefixes(S) ==
  LET P == UNION { Prefixes(seq) : seq \in S }
  IN { prefix \in P : \A t \in S: IsPrefix(prefix, t) }

(*****
(* The longest common prefix of the sequences in the set S.              *)
(*****
LongestCommonPrefix(S) ==
  CHOOSE longest \in CommonPrefixes(S): \* there can only be one LCP => CHOOSE
    \A other \in CommonPrefixes(S):
      Len(other) <= Len(longest)

(*****
(* The set of suffixes of the sequence s, including the empty sequence.  *)
(*****
Suffixes(s) ==
  { SubSeq(s, 1, Len(s)) : 1 \in 1..Len(s) } \cup {<<>}

-----

(*****
(* Range(a % b) = 0..b-1, but DOMAIN seq = 1..Len(seq).                *)
(* So to do modular arithmetic on sequences we need to                  *)
(* map 0 to b.                                                            *)
(*****
SeqMod(a, b) ==
  IF a % b = 0 THEN b ELSE a % b

```

```

(*****
(* An alias of FoldFunction that combines all elements of seq using op, *)
(* in an unspecified order. The result is *)
(* op(seq[i],op(seq[j], ..., op(seq[k],base) ...)) *)
(* *)
(* op should be associative and commutative, because we can not assume a *)
(* particular ordering of i, j, and k *)
(* *)
(* Example: *)
(* FoldSeq(LAMBDA x,y: {x} \cup y, {}, <<1,2,1>>) = Range(<<1,2,1>>) *)
(*****)
FoldSeq(op(_, _), base, seq) ==
  FoldFunction(op, base, seq)

(*****
(* FoldLeft folds op on all elements of seq from left to right, starting *)
(* with the first element and base. The resulting function is: *)
(* op(op(...op(base,f[0]), ...f[n-1]), f[n]) *)
(* *)
(* *)
(* Example: *)
(* LET cons(x,y) == <<x,y>> *)
(* IN FoldLeft(cons, 0, <<3,1,2>>) = << << <<0,3>>, 1>>, 2>> *)
(*****)
FoldLeft(op(_, _), base, seq) ==
  MapThenFoldSet(LAMBDA x,y : op(y,x), base,
    LAMBDA i : seq[i],
    LAMBDA S : Max(S),
    DOMAIN seq)

(*****
(* FoldRight folds op on all elements of seq from right to left, starting *)
(* with the last element and base. The resulting function is: *)
(* op(f[0],op(f[1], ..., op(f[n],base) ...)) *)
(* *)
(* *)
(* Example: *)
(* LET cons(x,y) == <<x,y>> *)
(* IN FoldRight(cons, <<3,1,2>>, 0 ) = << 3, << 1, <<2,0>> >> >> *)
(*****)
FoldRight(op(_, _), seq, base) ==
  MapThenFoldSet(op, base,
    LAMBDA i : seq[i],
    LAMBDA S : Min(S),
    DOMAIN seq)

(*****
(* FoldLeftDomain folds op on the domain of seq, i.e., the seq's indices, *)
(* starting at the lowest index. *)
(*****)
FoldLeftDomain(op(_, _), base, seq) ==
  FoldLeft(op, base, [i \in DOMAIN seq |-> i])

```

```

(*****
(* FoldRightDomain folds op on the domain of seq, i.e., the seq's indices, *)
(* starting at the highest index. *)
(*****
FoldRightDomain(op(_, _), seq, base) ==
  FoldRight(op, [i \in DOMAIN seq |-> i], base)

-----

(*****
(* A sequence of all elements from all sequences in the sequence seqs . *)
(* *)
(* Examples: *)
(* *)
(* FlattenSeq(<< <<1,2>>, <<1>> >>) = << 1, 2, 1 >> *)
(* FlattenSeq(<< <<"a">>, <<"b">> >>) = <<"a", "b">> *)
(* FlattenSeq(<< "a", "b" >>) = "ab" *)
(*****
FlattenSeq(seqs) ==
  IF Len(seqs) = 0 THEN seqs ELSE
    \* Not via FoldSeq(\o, <<>>, seqs) here to support strings with TLC.
    LET flatten[i \in 1..Len(seqs)] ==
      IF i = 1 THEN seqs[i] ELSE flatten[i-1] \o seqs[i]
    IN flatten[Len(seqs)]

(*****
(* A sequence of pairs where the i-th pair is formed from the i-th *)
(* element of s and the i-th element of t. The length of the result *)
(* sequence is the minimum of the lengths of s and t. *)
(* *)
(* Examples: *)
(* *)
(* Zip(<< >>, << >>) = << >> *)
(* Zip(<<"a">>, <<"b">>) = << <<"a", "b">> >> *)
(* Zip(<<1, 3>>, <<2, 4>>) = <<<<1, 2>>, <<3, 4>>>> *)
(* FlattenSeq(Zip(<<1, 3>>, <<2, 4>>)) = <<1, 2, 3, 4>>>> *)
(* Zip(<< >>, <<1, 2, 3>>) = << >> *)
(* Zip(<<"a", "b", "c">>, <<1>>) = << <<"a", 1>> >> *)
(*****
Zip(s, t) ==
  LET l == IF Len(s) <= Len(t) THEN Len(s) ELSE Len(t)
  IN [ i \in 1 .. l |-> <<s[i], t[i] >> ]

```

```

(*****)
(* A sequence where the i-th tuple contains the i-th element of s and *)
(* t in this order. Not defined for Len(s) # Len(t) *)
(* *)
(* Examples: *)
(* *)
(* Interleave(<< >>, << >>) = << >> *)
(* Interleave(<<"a">>, <<"b">>) = <<"a", "b">> *)
(* Interleave(<<1,3>>, <<2,4>>) = <<<<1>>, <<2>>, <<3>>, <<4>>>> *)
(* FlattenSeq(Interleave(<<1,3>>,<<2,4>>)) = <<1, 2, 3, 4>> *)
(*****)
Interleave(s, t) ==
CASE Len(s) = Len(t) /\ Len(s) > 0 ->
  LET u[ i \in 1..Len(s) ] ==
    IF i = 1 THEN << <s[i]>> >> \o << <t[i]>> >>
    ELSE u[i-1] \o << <s[i]>> >> \o << <t[i]>> >>
  IN Last(u)
  \* error "Interleave: sequences must have same length"
[] Len(s) = Len(t) /\ Len(s) = 0 -> << <<>>, <<>> >>

(*****)
(* The set of all subsequences of the sequence s . Note that the empty *)
(* sequence <<>> is defined to be a subsequence of any sequence. *)
(*****)
SubSeqs(s) ==
{ SubSeq(s, i+1, j) : i, j \in 0..Len(s) }

(*****)
(* SubSeqs(s) *)
(* \cup *)
(* { subsequences of s, with arbitrarily many elts removed } *)
(* *)
(* Example: *)
(* AllSubSeqs(<<1,2,3,4>>) = *)
(* {<<>>, <<1>>, <<2>>, <<3>>, <<4>>, *)
(* <<1, 2>>, <<1, 3>>, <<1, 4>>, *)
(* <<2, 3>>, <<2, 4>>, <<3, 4>>, *)
(* <<1, 2, 3>>, <<1, 2, 4>>, <<1, 3, 4>>, <<2, 3, 4>>, *)
(* <<1, 2, 3, 4>>} *)
(*****)
AllSubSeqs(s) ==
{ FoldFunction(Snoc, <<>>, [ i \in D |-> s[i] ]) : D \in SUBSET DOMAIN s }

(*****)
(* The (1-based) index of the beginning of the subsequence s of the *)
(* sequence t . If s appears in t multiple times, this equals the *)
(* lowest index. *)
(* For example: IndexFirstSubSeq(<<1>>, <<1,1,1>>) = 1 *)
(*****)
IndexFirstSubSeq(s, t) ==
LET last == CHOOSE i \in 0..Len(t) :
  /\ s \in SubSeqs(SubSeq(t, 1, i))
  /\ \A j \in 0..i-1 : s \notin SubSeqs(SubSeq(t, 1, j))
IN last - (Len(s) - 1)

```

```

(*****
(* The sequence t with its subsequence s at position i replaced by *)
(* the sequence r . *)
(*****)
ReplaceSubSeqAt(i, r, s, t) ==
  LET prefix == SubSeq(t, 1, i - 1)
    suffix == SubSeq(t, i + Len(s), Len(t))
  IN prefix \o r \o suffix

(*****
(* The sequence t with its subsequence s replaced by the sequence r *)
(*****)
ReplaceFirstSubSeq(r, s, t) ==
  IF s \in SubSeqs(t)
  THEN ReplaceSubSeqAt(IndexFirstSubSeq(s, t), r, s, t)
  ELSE t

(*****
(* The sequence t with all subsequences s replaced by the sequence r *)
(* Overlapping replacements are disambiguated by choosing the occurrence *)
(* closer to the beginning of the sequence. *)
(* *)
(* Examples: *)
(* *)
(* ReplaceAllSubSeqs(<<>>, <<>>, <<>>) = <<>> *)
(* ReplaceAllSubSeqs(<<4>>, <<>>, <<>>) = <<4>> *)
(* ReplaceAllSubSeqs(<<2>>, <<3>>, <<1,3>>) = <<1,2>> *)
(* ReplaceAllSubSeqs(<<2,2>>, <<1,1>>, <<1,1,1>>) = <<2,2,1>> *)
(*****)
ReplaceAllSubSeqs(r, s, t) ==
  CASE s = t -> r
  [] r = s -> t \* TLC optimization
  [] s # t /\ Len(s) = 0 ->
    LET z == Interleave([i \in 1..Len(t) |-> r], [i \in 1..Len(t) |-> <<t[i]>>])
    IN FlattenSeq(FlattenSeq(z))
  [] s # t /\ Len(s) > 0 /\ s \in SubSeqs(t) ->
    \* Not defined recursively to avoid infinite loops.
    LET match(f) == { i \in 1..Len(f) : s = f[i] }
      comp(p, q) == \A i \in 1..Len(p) : p[i] <= q[i]
      \* TODO: Replace with Seq(Seq(Range(t))) once *total* Java module
      \* override in place. The current override handles only the
      \* case where the parameters are strings (hence Range("abc")
      \* not a problem with TLC).
      R == BoundedSeq(BoundedSeq(Range(t), Len(t)), Len(t))
      \* A) Matches the input t.
      S == { f \in R : FlattenSeq(f) = t }
      \* B) Has the max number of matches...
      T == { f \in S : \A g \in S :
        Cardinality(match(g)) <= Cardinality(match(f)) }
      \* C) ...of min (leftmost) matches.
      u == CHOOSE f \in T :
        \A g \in T : comp(
          SetToSortSeq(match(f), <), SetToSortSeq(match(g), <))
        IN FlattenSeq([i \in 1..Len(u) |-> IF s = u[i] THEN r ELSE u[i]])
    [] OTHER -> t

```

```

=====

```

```

----- MODULE BagsExt -----
(*****
(* Additional operators on bags (multisets). *)
(* *)
(* Bags represent multisets - collections where elements can appear *)
(* multiple times. In TLA+, bags are represented as functions from *)
(* elements to positive natural numbers, where bag[x] gives the *)
(* multiplicity (count) of element x. *)
(* *)
(* This module extends the standard Bags module with: *)
(*   - Element manipulation (add, remove individual/all occurrences) *)
(*   - Folding operations (generalized aggregation over bag elements) *)
(*   - Arithmetic operations (sum, product of bag elements) *)
(* *)
(* Example bag representation: *)
(*   The bag {1, 1, 2, 3, 3, 3} is represented as: *)
(*   (1 :> 2) @@ (2 :> 1) @@ (3 :> 3) *)
(*****)

LOCAL INSTANCE Bags
LOCAL INSTANCE Integers
LOCAL INSTANCE Folds

(*****
(* Add one occurrence of element x to bag B. *)
(* *)
(* Increases the multiplicity of x by 1. If x is not in the bag, *)
(* adds it with multiplicity 1. *)
(* *)
(* This is equivalent to B (+) SetToBag({x}. *)
(* *)
(* Examples: *)
(*   BagAdd((1 :> 2) @@ (2 :> 1), 1) = (1 :> 3) @@ (2 :> 1) *)
(*   BagAdd((1 :> 2) @@ (2 :> 1), 3) = (1 :> 2) @@ (2 :> 1) @@ (3 :> 1) *)
(*****)

BagAdd(B, x) ==
  IF x \in DOMAIN B
  THEN [e \in DOMAIN B |> IF e=x THEN B[e]+1 ELSE B[e]]
  ELSE [e \in DOMAIN B \union {x} |> IF e=x THEN 1 ELSE B[e]]

```

```

(*****)
(* Remove one occurrence of element x from bag B. *)
(* *)
(* Decreases the multiplicity of x by 1. If x has multiplicity 1, *)
(* removes it entirely from the bag. If x is not in the bag, equals *)
(* the bag unchanged. *)
(* *)
(* This is equivalent to B (-) SetToBag({x}). *)
(* *)
(* Examples: *)
(* BagRemove((1 :> 2) @@ (2 :> 1), 1) = (1 :> 1) @@ (2 :> 1) *)
(* BagRemove((1 :> 1) @@ (2 :> 1), 1) = (2 :> 1) *)
(* BagRemove((1 :> 2) @@ (2 :> 1), 3) = (1 :> 2) @@ (2 :> 1) *)
(*****)
BagRemove(B,x) ==
  IF x \in DOMAIN B
  THEN IF B[x] = 1
    THEN [e \in DOMAIN B \ {x} |-> B[e]]
    ELSE [e \in DOMAIN B |-> IF e=x THEN B[e]-1 ELSE B[e]]
  ELSE B

(*****)
(* Remove all occurrences of element x from bag B. *)
(* *)
(* Completely removes element x from the bag, regardless of its *)
(* multiplicity. If x is not in the bag, returns the bag unchanged. *)
(* *)
(* Examples: *)
(* BagRemoveAll((1 :> 3) @@ (2 :> 1), 1) = (2 :> 1) *)
(* BagRemoveAll((1 :> 3) @@ (2 :> 1), 3) = (1 :> 3) @@ (2 :> 1) *)
(*****)
BagRemoveAll(B,x) ==
  [e \in DOMAIN B \ {x} |-> B[e]]

```

```

(*****)
(* Fold operation op over the images through f of all elements of bag *)
(* B, starting from base. The parameter choose indicates the order in *)
(* which elements of the bag are processed; all replicas of an element *)
(* are processed consecutively. *)
(* *)
(* Examples: *)
(*   MapThenFoldBag(LAMBDA x,y : x+y, *)
(*                   0, *)
(*                   LAMBDA x: 1, *)
(*                   LAMBDA B : CHOOSE x \in DOMAIN B : TRUE, *)
(*                   (1 :> 2) @@ (2 :> 1) @@ (3 :> 3)) = 6 *)
(*   MapThenFoldBag(LAMBDA x,y : x \o y, *)
(*                   << >>, *)
(*                   LAMBDA x: << x >>, *)
(*                   LAMBDA S: CHOOSE x \in S : \A y \in S : x <= y, *)
(*                   (1 :> 2) @@ (2 :> 1) @@ (3 :> 3)) *)
(*                   = <<1,1,2,3,3,3>> *)
(* *)
(* The fifth argument represents the bag {| 1, 1, 2, 3, 3, 3 |}. *)
(* The first example counts the elements of the bag. *)
(* The second example computes a sorted sequence of all elements *)
(* of the bag. It is brittle because concatenation of sequences is *)
(* non-commutative, and the result therefore relies on the *)
(* definition of MapThenFoldSet. *)
(*****)
MapThenFoldBag(op(_,_), base, f(_), choose(_), B) ==
  LET handle(x) == \* handle all occurrences of x in B
    LET pow[n \in Nat \ {0}] ==
      op(IF n=1 THEN base ELSE pow[n-1], f(x))
    IN pow[B[x]]
  IN MapThenFoldSet(op, base, handle,
    LAMBDA S : CHOOSE x \in S : TRUE,
    DOMAIN B)

(*****)
(* Fold op over all elements of bag B, starting with value base. *)
(* *)
(* Example: *)
(*   FoldBag(LAMBDA x,y : x+y, *)
(*           0, *)
(*           (1 :> 2) @@ (2 :> 1) @@ (3 :> 3)) = 13 *)
(* The third argument represents the bag {| 1, 1, 2, 3, 3, 3 |}. *)
(*****)
FoldBag(op(_,_), base, B) ==
  MapThenFoldBag(op, base, LAMBDA x: x, LAMBDA S : CHOOSE x \in S : TRUE, B)

(*****)
(* Compute the sum of the elements of B. *)
(*****)
SumBag(B) ==
  FoldBag(LAMBDA x,y : x+y, 0, B)

```

```

(*****
(* Compute the product of the elements of B. *)
(*****)
ProductBag(B) ==
  FoldBag(LAMBDA x,y : x*y, 1, B)

=====

----- MODULE Graphs -----
LOCAL INSTANCE Naturals
LOCAL INSTANCE Sequences
LOCAL INSTANCE SequencesExt
LOCAL INSTANCE FiniteSets
LOCAL INSTANCE Relation

IsDirectedGraph(G) ==
  /\ G = [node |-> G.node, edge |-> G.edge]
  /\ G.edge \subsepeq (G.node \X G.node)

DirectedSubgraph(G) ==
  {H \in [node : SUBSET G.node, edge : SUBSET (G.node \X G.node)] :
    IsDirectedGraph(H) /\ H.edge \subsepeq G.edge}

Transpose(G) ==
  \* https://en.wikipedia.org/wiki/Transpose_graph
  [ edge |-> { <<e[2], e[1]>> : e \in G.edge },
    node |-> G.node]

-----

IsUndirectedGraph(G) ==
  /\ IsDirectedGraph(G)
  /\ \A e \in G.edge : <<e[2], e[1]>> \in G.edge

UndirectedSubgraph(G) == {H \in DirectedSubgraph(G) : IsUndirectedGraph(H)}

-----

Path(G) == {p \in Seq(G.node) :
  /\ p # << >>
  /\ \A i \in 1..(Len(p)-1) : <<p[i], p[i+1]>> \in G.edge}

SimplePath(G) ==
  \* A simple path is a path with no repeated nodes.
  {p \in SeqOf(G.node, Cardinality(G.node)) :
    /\ p # << >>
    /\ Cardinality({ p[i] : i \in DOMAIN p }) = Len(p)
    /\ \A i \in 1..(Len(p)-1) : <<p[i], p[i+1]>> \in G.edge}

AreConnectedIn(m, n, G) ==
  \E p \in SimplePath(G) : (p[1] = m) /\ (p[Len(p)] = n)

```

```

ConnectionsIn(G) ==
  \* Compute a Boolean matrix that indicates, for each pair of nodes,
  \* if there exists a path that links the two nodes. The computation,
  \* based on Warshall's algorithm, is much more efficient than the
  \* definition used in AreConnectedIn, and the result can be cached
  \* by TLC, avoiding recomputation.
  LET C[N \in SUBSET G.node] ==
    \* Matrix representing the existence of paths whose inner nodes
    \* (i.e., except for the source and the target) are all in the
    \* set of nodes N.
    IF N = {}
    THEN [m,n \in G.node |-> m = n \/\ <<m,n>> \in G.edge]
    ELSE LET u == CHOOSE u \in N : TRUE
          Cu == C[N \ {u}]
          IN [m,n \in G.node |-> \/\ Cu[m,n]
              \/\ Cu[m,u] /\ Cu[u,n]]

  IN C[G.node]

IsStronglyConnected(G) ==
  \A m, n \in G.node : AreConnectedIn(m, n, G)
-----
IsTreeWithRoot(G, r) ==
  /\ IsDirectedGraph(G)
  /\ \A e \in G.edge : /\ e[1] # r
                        /\ \A f \in G.edge : (e[1] = f[1]) => (e = f)
  /\ \A n \in G.node : AreConnectedIn(n, r, G)
-----
(*****
(* Returns the union of two graphs. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2}, edge |-> {<<1, 2>>}] *)
(* H = [node |-> {2, 3}, edge |-> {<<2, 3>>}] *)
(* GraphUnion(G, H) *)
(* = [node |-> {1, 2, 3}, edge |-> {<<1, 2>>, <<2, 3>>}] *)
(*****
GraphUnion(G, H) ==
  [node |-> G.node \union H.node, edge |-> G.edge \union H.edge]

(*****
(* Checks whether the graph G is bipartite with partitions U and V. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3, 4}, *)
(* edge |-> {<<1, 2>>, <<2, 3>>, <<3, 4>>}] *)
(* IsBipartiteWithPartitions(G, {1, 3}, {2, 4}) = TRUE *)
(*****
IsBipartiteWithPartitions(G, U, V) ==
  /\ U \cap V = {}
  /\ G.node \subseq (U \cup V)
  /\ \A e \in G.edge: \/\ e[1] \in U /\ e[2] \in V
                    \/\ e[2] \in U /\ e[1] \in V

```

```

(*****
(* Checks whether the graph G contains a cycle. *)
(* *)
(* Note: Relies on the definition of ConnectionsIn. Please note that this *)
(* operator is defined recursively. *)
(*****
HasCycle(G) ==
  \/\ E n \in G.node: << n, n >> \in G.edge
  \/\ E m, n \in G.node:
    /\ m # n
    /\ ConnectionsIn(G)[m, n]
    /\ ConnectionsIn(G)[n, m]

(*****
(* Checks whether the directed graph G is a directed Acyclic Graph (DAG). *)
(*****
IsDag(G) ==
  /\ IsDirectedGraph(G)
  /\ ~HasCycle(G)

(*****
(* Returns the set of nodes that are immediate successors of node n in G. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<1, 2>>, <<1, 3>>}] *)
(* Successors(G, 1) = {2, 3} *)
(*****
Successors(G, n) == {m \in G.node: << n, m >> \in G.edge}

(*****
(* Returns the set of nodes that are immediate successors of any node in S. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<1, 2>>, <<1, 3>>}] *)
(* AllSuccessors(G, {1, 2}) = {2, 3} *)
(*****
AllSuccessors(G, S) == UNION {Successors(G, n): n \in S}

(*****
(* Returns the set of nodes that are immediate predecessors of node n in G. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<2, 1>>, <<3, 1>>}] *)
(* Predecessors(G, 1) = {2, 3} *)
(*****
Predecessors(G, n) == {m \in G.node: << m, n >> \in G.edge}

(*****
(* Returns the set of nodes that are immediate predecessors of any node in S. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<2, 1>>, <<3, 1>>}] *)
(* AllPredecessors(G, {1, 2}) = {2, 3} *)
(*****
AllPredecessors(G, S) == UNION {Predecessors(G, n): n \in S}

```

```

(*****)
(* Returns the set of all nodes in graph G that have a path to node n. *)
(* *)
(* Example: *)
(* G == [node |-> {1, 2, 3, 4}, edge |-> {<<4, 2>>, <<2, 1>>, <<3, 1>>}] *)
(* Ancestors(G, 1) = {2, 3, 4} *)
(*****)
Ancestors(G, n) ==
  LET EdgeRelation ==
    [<<x, y>> \in G.node \X G.node |-> <<x, y>> \in G.edge]
  IN { m \in G.node : TransitiveClosure(EdgeRelation, G.node)[m, n] }

(*****)
(* Returns the set of all nodes in graph G that are reachable from node n *)
(* *)
(* Example: *)
(* G == [node |-> {1, 2, 3, 4}, edge |-> {<<4, 2>>, <<2, 1>>, <<3, 1>>}] *)
(* Descendants(G, 4) = {1, 2} *)
(*****)
Descendants(G, n) ==
  LET EdgeRelation ==
    [<<x, y>> \in G.node \X G.node |-> <<x, y>> \in G.edge]
  IN { m \in G.node : TransitiveClosure(EdgeRelation, G.node)[n, m] }

(*****)
(* Returns the in-degree of node n in directed graph G. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<2, 1>>, <<3, 1>>}] *)
(* InDegree(G, 1) = 2 *)
(*****)
InDegree(G, n) == Cardinality(Predecessors(G, n))

(*****)
(* Returns the out-degree of node n in directed graph G. *)
(* *)
(* Example: *)
(* G = [node |-> {1, 2, 3}, edge |-> {<<1, 2>>, <<1, 3>>}] *)
(* OutDegree(G, 1) = 2 *)
(*****)
OutDegree(G, n) == Cardinality(Successors(G, n))

(*****)
(* Returns the set of root nodes of G. *)
(*****)
Roots(G) == {n \in G.node: Predecessors(G, n) = {}}

(*****)
(* Returns the set of leaf nodes of G. *)
(*****)
Leaves(G) == {n \in G.node: Successors(G, n) = {}}

-----
(*****)
(* The graph with no nodes and no edges. *)
(*****)
EmptyGraph == [node |-> {}, edge |-> {}]

```

```

(*****
(* The set of all possible labeled directed graphs whose node set is S. *)
(*
(* Example:
(*   Graphs({1, 2}) = {
(*     [node |-> {1, 2}, edge |-> {}],
(*     [node |-> {1, 2}, edge |-> {<<1, 2>>}],
(*     [node |-> {1, 2}, edge |-> {<<1, 2>>, <<2, 1>>}],
(*     ...
(*   }
(*****
Graphs(S) == [node: {S}, edge: SUBSET (S \X S)]
=====
\* Modification History
\* Last modified Sun Mar 06 18:10:34 CET 2022 by Stephan Merz
\* Last modified Tue Dec 21 15:55:45 PST 2021 by Markus Kuppe
\* Created Tue Jun 18 11:44:08 PST 2002 by Leslie Lamport

----- MODULE UndirectedGraphs -----
(*****
(* Representation of undirected graphs in TLA+. In contrast to module *)
(* Graphs, edges are represented as unordered pairs {a,b} of nodes, thus *)
(* enforcing symmetry. *)
(*****
LOCAL INSTANCE Naturals
LOCAL INSTANCE Sequences
LOCAL INSTANCE SequencesExt
LOCAL INSTANCE FiniteSets
LOCAL INSTANCE Folds

IsUndirectedGraph(G) ==
  /\ G = [node |-> G.node, edge |-> G.edge]
  /\ \A e \in G.edge : \E a,b \in G.node : e = {a,b}

IsLoopFreeUndirectedGraph(G) ==
  /\ G = [node |-> G.node, edge |-> G.edge]
  /\ \A e \in G.edge : \E a,b \in G.node : a # b /\ e = {a,b}

UndirectedSubgraph(G) ==
  {H \in [node : SUBSET G.node, edge : SUBSET G.edge] : IsUndirectedGraph(H)}

-----
Path(G) == {p \in Seq(G.node) :
  /\ p # << >>
  /\ \A i \in 1..(Len(p)-1) : {p[i], p[i+1]} \in G.edge}

SimplePath(G) ==
  \* A simple path is a path with no repeated nodes.
  {p \in SeqOf(G.node, Cardinality(G.node)) :
    /\ p # << >>
    /\ Cardinality({ p[i] : i \in DOMAIN p }) = Len(p)
    /\ \A i \in 1..(Len(p)-1) : {p[i], p[i+1]} \in G.edge}

```

```

(*****
(* Compute the connected components of an undirected graph: initially each *)
(* node is in a component by itself, then iterate over the edges to merge *)
(* the components related by the edge. *)
(*****)
ConnectedComponents(G) ==
  LET base == {{n} : n \in G.node}
  choice(E) == CHOOSE e \in E : TRUE
  firstNode(e) == CHOOSE a \in G.node : \E b \in G.node : e = {a,b}
  secondNode(e) == CHOOSE b \in G.node : e = {firstNode(e), b}
  nodesOfEdge(e) == <<firstNode(e), secondNode(e)>>
  merge(e, comps) ==
    LET compA == CHOOSE c \in comps : e[1] \in c
    compB == CHOOSE c \in comps : e[2] \in c
    IN IF compA = compB THEN comps
    ELSE (comps \ {compA, compB}) \union {compA \union compB}
  IN MapThenFoldSet(merge, base, nodesOfEdge, choice, G.edge)

AreConnectedIn(m, n, G) ==
  \E comp \in ConnectedComponents(G) : m \in comp /\ n \in comp

IsStronglyConnected(G) ==
  Cardinality(ConnectedComponents(G)) = 1
=====
----- MODULE GraphViz -----
DotDiGraph(G, VL(_), EL(_)) ==
  TRUE
=====
----- MODULE Bitwise -----
LOCAL INSTANCE Integers

\* https://en.wikipedia.org/wiki/Bitwise_operation#Mathematical_equivalents
RECURSIVE And(_,_,_,_ )
LOCAL And(x,y,n,m) ==
  LET exp == 2^n
  IN IF m = 0
  THEN 0
  ELSE exp * ((x \div exp) % 2) * ((y \div exp) % 2)
    + And(x,y,n+1,m \div 2)

(*****
(* Bitwise AND of (non-negative) x and y (defined for Nat \cup {0}). *)
(*****)
x & y ==
  IF x >= y THEN And(x, y, 0, x) ELSE And(y, x, 0, y) \* Infix variant of And(x,y)
-----

RECURSIVE Or(_,_,_,_ )
LOCAL Or(x,y,n,m) ==
  LET exp == 2^n
  xdm == (x \div exp) % 2
  ydm == (y \div exp) % 2
  IN IF m = 0
  THEN 0
  ELSE exp * (((xdm + ydm) + (xdm * ydm)) % 2)
    + Or(x,y,n+1,m \div 2)

```

```

(*****
(* Bitwise OR of (non-negative) x and y (defined for Nat \cup {0}).      *)
(*****
x | y ==
  IF x >= y THEN Or(x, y, 0, x) ELSE Or(y, x, 0, y) \* Infix variant of Or(x,y)

-----

RECURSIVE Xor(_,_,_)
LOCAL Xor(x,y,n,m) ==
  LET exp == 2^n
  IN IF m = 0
    THEN 0
    ELSE exp * (((x \div exp) + (y \div exp)) % 2)
      + Xor(x,y,n+1,m \div 2)

(*****
(* Bitwise XOR of (non-negative) x and y (defined for Nat \cup {0}).      *)
(*****
x ^ y == \* single "^" already taken by Naturals.tla
  IF x >= y THEN Xor(x, y, 0, x) ELSE Xor(y, x, 0, y) \* Infix variant of Xor(x,y)

-----

RECURSIVE NotR(_,_,_)
LOCAL NotR(x,n,m) ==
  LET exp == 2^n
  xdm == (x \div exp) % 2
  IN IF m = 0
    THEN 0
    ELSE exp * ((xdm + 1) % 2)
      + NotR(x,n+1,m \div 2)

(*****
(* Bitwise NOT of (non-negative) x (defined for Nat \cup {0}).          *)
(*****
Not(a) ==
  NotR(a,0,a)

-----

(*****
(* Logical bit-shifting the (non-negative) n by pos positions to the right *)
(* shifting zeros in from the left/MSB (defined for Nat \cup {0}).        *)
(*****
RECURSIVE shiftR(_,_)
shiftR(n,pos) ==
  IF pos = 0
    THEN n
    ELSE LET odd(z) == z % 2 = 1
      m == IF odd(n) THEN (n-1) \div 2 ELSE n \div 2
      IN shiftR(m, pos - 1)

=====
\* Modification History
\* Last modified Thu April 25 10:56:12 CEST 2018 by markus
\* Created Mon May 16 15:09:18 CEST 2016 by markus

----- MODULE Combinatorics -----
EXTENDS Naturals

```

```

factorial[n \in Nat] ==
  IF n = 0 THEN 1 ELSE n * factorial[n-1]

choose(n, k) ==
  factorial[n] \div (factorial[k] * factorial[n - k])

=====

----- MODULE CSV -----
(* ***** *)
(* This module provides operators for reading and writing CSV (Comma *)
(* Separated Values) files. These operators are overridden by Java *)
(* implementations that perform the actual file I/O operations. *)
(* *)
(* The CSV module is particularly useful for: *)
(*   - Logging TLA+ values during model checking *)
(*   - Exporting computation results for analysis *)
(*   - Creating test data files *)
(*   - Interfacing with external data processing tools *)
(* *)
(* Note: All operators in this module evaluate to TRUE in TLA+ but *)
(* perform side effects when executed by TLC with the Java overrides. *)
(* ***** *)

LOCAL INSTANCE TLC
LOCAL INSTANCE Integers
  (* ***** *)
  (* Imports the definitions from the modules, but doesn't export them. *)
  (* ***** *)

  (* ***** *)
  (* Writes a sequence of values to a CSV file using a format template. *)
  (* *)
  (* Parameters: *)
  (*   - template: Printf-style format string with placeholders *)
  (*   - val: Sequence of values to write *)
  (*   - file: Absolute path to the output file *)
  (* *)
  (* The template uses Java's String.format syntax: *)
  (*   - %1$s refers to the first element in val *)
  (*   - %2$s refers to the second element in val, etc. *)
  (* *)
  (* Example: *)
  (*   CSVWrite("%1$s#%2$s#%3$s", *)
  (*     <<"abc", 42, {"x", "y"}>>, *)
  (*     "/tmp/output.csv") *)
  (*   Writes: "abc#42#{\"x\", \"y\"}" to /tmp/output.csv *)
  (* ***** *)
  CSVWrite(template, val, file) ==
    TRUE

```

```

(*****)
(* Writes a record to a CSV file with optional headers. *)
(* *)
(* Parameters: *)
(* - record: A record (function) to write *)
(* - delim: Field delimiter string *)
(* - headers: Boolean indicating whether to write headers *)
(* - file: Absolute path to the output file *)
(* *)
(* If headers is TRUE and the file is empty, field names are written *)
(* as the first row. The record values are then written as a data row. *)
(* *)
(* Example: *)
(* CSVWriteRecord([name |-> "Alice", age |-> 30], *)
(*      ",", *)
(*      CSVRecords("/tmp/people.csv") = 0, *)
(*      "/tmp/people.csv") *)
(* First call writes: "name,age" then "Alice,30" *)
(* Subsequent calls write only data rows *)
(*****)
CSVWriteRecord(record, delim, headers, file) ==
  TRUE

(*****)
(* Reads a CSV file and returns the data as a sequence of records. *)
(* *)
(* Parameters: *)
(* - columns: Sequence of column names for the records *)
(* - delimiter: Field delimiter character *)
(* - file: Absolute path to the input file *)
(* *)
(* Returns a sequence where each element is a record mapping column *)
(* names to their string values. All values are strings and must be *)
(* converted if numeric processing is needed. *)
(* *)
(* Example: *)
(* CSVRead(<<"name", "age", "city">>, ",", "/tmp/people.csv") *)
(* might return: *)
(* <<[name |-> "Alice", age |-> "30", city |-> "NYC"], *)
(*      [name |-> "Bob", age |-> "25", city |-> "LA"]>> *)
(*****)
CSVRead(columns, delimiter, file) ==
  TRUE

(*****)
(* Returns the number of records in a CSV file. *)
(* *)
(* This includes all rows in the file, including any header row. *)
(* For files that don't exist, returns 0. *)
(* *)
(* Example: *)
(* CSVRecords("/tmp/data.csv") = 5 \* File has 5 rows *)
(* CSVRecords("/nonexistent.csv") = 0 *)
(*****)
CSVRecords(file) ==
  CHOOSE n : n \in Nat
=====

```

```

----- MODULE DifferentialEquations -----
LOCAL INSTANCE Reals
LOCAL INSTANCE Sequences
LOCAL PosReal == {r \in Real : r > 0}
LOCAL OpenInterval(a, b) == {s \in Real : a < s /\ s < b}
LOCAL Nbhd(r,e) == OpenInterval(r-e, r+e)
LOCAL IsFirstDeriv(df, f) ==
  /\ df \in [DOMAIN f -> Real]
  /\ \A r \in DOMAIN f :
    \A e \in PosReal :
      \E d \in PosReal :
        \A s \in Nbhd(r,d) \ {r} :
          (f[s] - f[r])/(s - r) \in Nbhd(df[r], e)

LOCAL IsDeriv(n, df, f) ==
  LET IsD[k \in 0..n, g \in [DOMAIN f -> Real]] ==
    IF k = 0
      THEN g = f
      ELSE \E gg \in [DOMAIN f -> Real] : /\ IsFirstDeriv(g, gg)
    /\ IsD[k-1, gg]

  IN IsD[n, df]

Integrate(D, a, b, InitVals) ==
  LET n == Len(InitVals)
  gg == CHOOSE g :
    \E e \in PosReal :
      /\ g \in [0..n -> [OpenInterval(a-e, b+e) -> Real]]
      /\ \A i \in 1..n : /\ IsDeriv(i, g[i], g[0])
      /\ g[i-1][a] = InitVals[i]
      /\ \A r \in OpenInterval(a-e, b+e) :
        D[ <<r>> \o [i \in 1..(n+1) -> g[i-1][r]] ] = 0
  IN [i \in 1..n -> gg[i-1][b]]
=====
----- MODULE DyadicRationals -----
\* https://en.wikipedia.org/wiki/Dyadic_rational
LOCAL INSTANCE FiniteSets
LOCAL INSTANCE Integers
LOCAL INSTANCE Sequences
LOCAL INSTANCE FiniteSetsExt
LOCAL INSTANCE TLC

LOCAL Divides(p, q) ==
  \E d \in 1..q : q = p * d

LOCAL Divisors(q) ==
  {d \in 1..q : Divides(d, q)}

LOCAL GCD(n, m) ==
  Max(Divisors(n) \cap Divisors(m))
-----

LOCAL Rational(num, den) ==
  [num -> num, den -> den]

LOCAL Reduce(p) ==
  LET gcd == GCD(p.num, p.den)
  IN IF gcd = 1 THEN p
    ELSE Rational(p.num \div gcd, p.den \div gcd)

```

```

IsDyadicRational(r) ==
  \E i \in 0..r.den: 2^i = r.den

Zero == Rational(0,1)

One == Rational(1,1)

Add(p, q) ==
  IF p = Zero THEN q ELSE
  LET lcn == Max({p.den, q.den}) \* shortcut because dyadic!
  qq == Rational(q.num * (lcn \div q.den), q.den * (lcn \div q.den))
  pp == Rational(p.num * (lcn \div p.den), p.den * (lcn \div p.den))
  IN Reduce(Rational(qq.num + pp.num, lcn))

Half(p) ==
  Reduce(Rational(p.num, p.den * 2))

PrettyPrint(p) ==
  IF p = Zero THEN "0" ELSE
  IF p = One THEN "1" ELSE
  ToString(p.num) \o "/" \o ToString(p.den)

=====
\* Modification History
\* Last modified Mon Dec 27 17:52:54 PST 2021 by Markus Kuppe
\* Created Sun Dec 26 01:36:40 PDT 2021 by Markus Kuppe
----- MODULE HTML -----

LOCAL INSTANCE Sequences
LOCAL INSTANCE SequencesExt

\**** Helpers

HTMLFill(string, args) ==
  (*****
  (* Do the sequence of replaces in args to the string. *)
  (* The args is a sequence of tuples: <<substr,newsubstr>>. *)
  (* Example: *)
  (* HTMLFill("test %num%", <<"%num%", "12">> >>) = "test 12" *)
  (*****
  LET f(str, x) == ReplaceFirstSubSeq(x[2], x[1], str)
  IN FoldLeft(f, string, args)

LOCAL StringSeqToString(string_seq, separator) ==
  (*****
  (* Concatenates sequence of strings into one string. *)
  (* The 'separator' is put between the strings in the sequence. *)
  (* Example: *)
  (* StringSeqToString(<<"Test", "string!">>, "-") = "Test-string!" *)
  (*****
  LET f(str1, str2) == IF str1 = "" THEN str2 ELSE str1 \o separator \o str2
  IN FoldLeft(f, "", string_seq)

\**** Main Document

```

```

HTMLDocument(header, body, list_of_scripts) ==
    StringSeqToString(<<
        "<!DOCTYPE html>",
        "<html>",
        header,
        body,
        StringSeqToString(list_of_scripts, "\n"),
        "</html>"
    >>, "\n")

\***** Style Sheet

HTMLClass(classname, attribute_list) ==
    StringSeqToString(<<
        ReplaceFirstSubSeq(classname, "%classname%", "%classname% {"),
        StringSeqToString(attribute_list, "\n"),
        "}"
    >>, "\n")

HTMLAttribute(name, value) ==
    HTMLFill("%name%: %value%;", <<
        <<"%name%", name>>,
        <<"%value%", value>>
    >>)

HTMLDefaultStyle ==
    StringSeqToString(<<
        HTMLClass(".default_grid", <<
            HTMLAttribute("display", "grid")
        >>),
        HTMLClass(".default_box", <<
            HTMLAttribute("border", "1px solid black")
        >>),
        HTMLClass(".default_circle", <<
            HTMLAttribute("border-radius", "50%")
        >>)
    >>, "\n")

HTMLStyleSheet(class_list) ==
    StringSeqToString(<<
        "<style type=\"text/css\">",
        HTMLDefaultStyle,
        StringSeqToString(class_list, "\n"),
        "</style>"
    >>, "\n")

\***** Script

HTMLScriptFile(name) ==
    HTMLFill("<script src=\"%name%\"></script>", << <<"%name%", name>>>> )

HTMLScript(children) ==
    StringSeqToString(<<
        "<script>",
        StringSeqToString(children, "\n"),
        "</script>"
    >>, "\n")

\***** Header

```

```

HTMLLink(name, type) ==
    HTMLFill("<link href=\"%name%\" rel=\"%type%\"/>",
        << <<"%name%", name>>, <<"%type%", type>> >> )

HTMLHeader(title, list_of_links, class_list) ==
    StringSeqToString(<<
        "<head>",
        "<meta charset=\"UTF-8\">",
        ReplaceFirstSubSeq(title, "%title%", "<title>%title%</title>"),
        StringSeqToString(list_of_links, "\n"),
        HTMLStyleSheet(class_list),
        "</head>"
    >>, "\n")

\***** Body

HTMLBody(body) ==
    StringSeqToString(<<
        "<body>",
        body,
        "</body>"
    >>, "\n")

HTMLFrame(id, children) ==
    StringSeqToString(<<
        ReplaceFirstSubSeq(id, "%id%", "<div id=\"%id%\">"),
        StringSeqToString(children, "\n"),
        "</div>"
    >>, "\n")

HTMLGridContainer(id, class_list, children) ==
    StringSeqToString(<<
        HTMLFill("<div id=\"%id%\" class=\"default_grid %class_list%\">", <<
            <<"%id%", id>>,
            <<"%class_list%", StringSeqToString(class_list, " ")>>
        >>),
        StringSeqToString(children, "\n"),
        "</div>"
    >>, "\n")

HTMLBox(id, class_list, size, children) ==
    StringSeqToString(<<
        HTMLFill("<div id=\"%id%\" class=\"default_box %class_list%\" style=\"%size%\">", <<
            <<"%id%", id>>,
            <<"%class_list%", StringSeqToString(class_list, " ")>>,
            <<"%size%", size>>
        >>),
        StringSeqToString(children, "\n"),
        "</div>"
    >>, "\n")

HTMLCircle(id, class_list, size, children) ==
    HTMLBox(id, <<"default_circle">> \o class_list, size, children)

HTMLText(id, text) ==
    StringSeqToString(<<
        ReplaceFirstSubSeq(id, "%id%", "<span id=\"%id%\">"),
        text,
        "</span>"
    >>, "\n")

```

```

HTMLSVG(id, viewBox, size, svgString) ==
  StringSeqToString(<<
    HTMLFill("<svg id=\"%id%\" viewBox=\"%viewBox%\" style=\"%size%\">", <<
      <<"%id%", id>>,
      <<"%viewBox%", viewBox>>,
      <<"%size%", size>>
    >>),
    svgString,
    "</svg>"
  >>, "\n")

\***** Pre built

HTMLSize(width, height) ==
  StringSeqToString(<<
    HTMLAttribute("width", width),
    HTMLAttribute("height", height)
  >>, " ")

HTMLNewLine ==
  "<br>"

HTMLFlexCenterAttributes ==
  StringSeqToString(<<
    HTMLAttribute("display", "flex"),
    HTMLAttribute("align-items", "center"),
    HTMLAttribute("justify-content", "center")
  >>,"\\n")

HTMLJSLineElems(fromID, destID) ==
  HTMLFill("document.getElementById(\"%id1%\"), document.getElementById(\"%id2%\"),
    << <<"%id1%", fromID>>, <<"%id2%", destID>> >> )

HTMLJSLine(fromID, destID, color, size) ==
  HTMLFill("new LeaderLine(%elems%, {color: '%color%', size: %size%})",
    << <<"%elems%", HTMLJSLineElems(fromID, destID)>>,
    <<"%color%", color>>, <<"%size%", size>> >> )

HTMLJSLineDash(fromID, destID, color, size) ==
  HTMLFill("new LeaderLine(%elems%, {color: '%color%', size: %size%, dash: true})",
    << <<"%elems%", HTMLJSLineElems(fromID, destID)>>,
    <<"%color%", color>>, <<"%size%", size>> >> )

HTMLJSKeyListener(events) ==
  StringSeqToString(<<
    "window.onkeyup = function(event) {",
    "let key = event.key.toUpperCase();",
    StringSeqToString(events,"\\n"),
    "}"
  >>, "\n")

HTMLJSNavigationKey(key, file) ==
  StringSeqToString(<<
    ReplaceFirstSubSeq(key, "%key%", "if(key == '%key%'){",
    ReplaceFirstSubSeq(file, "%file%", "window.location.replace(\"%file%\");"),
    "}"
  >>,"\\n")
=====

```

```

----- MODULE IOUtils -----
(*****
(* This module provides operators for input/output operations and *)
(* external process execution. All operators are overridden by Java *)
(* implementations that perform the actual I/O operations. *)
(* *)
(* The module enables TLA+ specifications to: *)
(* - Execute external commands and capture their output *)
(* - Serialize/deserialize TLA+ values to/from files *)
(* - Access environment variables *)
(* - Interface with external systems during model checking *)
(* *)
(* Note: Some operators in this module evaluate to TRUE in TLA+ but *)
(* perform side effects when executed by TLC with the Java overrides. *)
(*****)

LOCAL INSTANCE TLC
LOCAL INSTANCE Integers
LOCAL INSTANCE Sequences
LOCAL INSTANCE SequencesExt
(*****
(* Imports the definitions from the modules, but doesn't export them. *)
(*****)

(*****
(* Serializes the given value to the file in Java's object serialization *)
(* format. *)
(* *)
(* Parameters: *)
(* - val: The TLA+ value to serialize *)
(* - absoluteFilename: Absolute path to the output file *)
(* - compress: Boolean indicating whether to compress the output *)
(*****)
IOSerialize(val, absoluteFilename, compress) == TRUE

(*****
(* Deserializes a TLA+ value from a file in Java's object serialization *)
(* format. *)
(* *)
(* Parameters: *)
(* - absoluteFilename: Absolute path to the input file *)
(* - compressed: Boolean indicating whether the file is compressed *)
(* *)
(* Returns the deserialized TLA+ value. *)
(*****)
IODeserialize(absoluteFilename, compressed) == CHOOSE val : TRUE

(*****
(* value: TLA+ value to be serialized. *)
(* dest: Destination to serialize to such as a file or URL. *)
(* options: Record of serializer-specific options with format mandatory to *)
(* identify a serializer. Read a serialzier's documentation for serializer- *)
(* specific options. *)
(*****)
Serialize(value, dest, options) ==
  CHOOSE r \in [exitValue : Int, stdout : STRING, stderr : STRING] : TRUE

```

```

(*****)
(* src: Destination to serialize to such as a file or URL. *)
(* options: Record of serializer-specific options with format mandatory to *)
(* identify a serializer. Read a seriazlier's documentation for serializer- *)
(* specific options. *)
(*****)
Deserialize(src, options) ==
    CHOOSE val : TRUE

-----

(*****)
(* Spawns the given command as a sub-process of TLC. The sequence of sequence *)
(* of strings 'command' signifies the external program file to be invoked and *)
(* its arguments: *)
(* IOExec(<<"ls", "-lah", "/tmp">>) *)
(* IOExec(<<"bash", "-c", "echo \"Col1#Col2\" > /tmp/out.csv">>) *)
(* see https://docs.oracle.com/javase/7/docs/api/java/lang/ProcessBuilder.html *)
(*****)
IOExec(command) ==
    CHOOSE r \in [exitValue : Int, stdout : STRING, stderr : STRING] : TRUE

(*****)
(* See IOExec *)
(* LET ENV = {<<"Var1", "SomeVal">>, <<"Var2", 42>>} *)
(* IN IOEnvExec(ENV, <<"ls", "-lah", "/tmp">>) *)
(*****)
IOEnvExec(env, command) ==
    CHOOSE r \in [exitValue : Int, stdout : STRING, stderr : STRING] : TRUE

(*****)
(* Spawns the given printf-style command as a sub-process of TLC. The *)
(* n-th flag in 'commandTemplate' is substituted with the n-th element *)
(* of the sequence 'parameters': *)
(* IOExecTemplate(<<"ls", "%s" "%s">>, <<"-lah", "/tmp">>) *)
(* IOExecTemplate(<<"ls %1$s %2$s">>, <<"-lah", "/tmp">>) *)
(* see http://docs.oracle.com/javase/7/docs/api/java/util/Formatter.html *)
(*****)
IOExecTemplate(commandTemplate, parameters) ==
    CHOOSE r \in [exitValue : Int, stdout : STRING, stderr : STRING] : TRUE

(*****)
(* See IOExecTemplate *)
(* LET ENV = {<<"Var1", "SomeVal">>, <<"Var2", 42>>} *)
(* IN IOEnvExecTemplate(ENV, <<"ls", "-lah", "/tmp">>) *)
(*****)
IOEnvExecTemplate(env, commandTemplate, parameters) ==
    CHOOSE r \in [exitValue : Int, stdout : STRING, stderr : STRING] : TRUE

(*****)
(* The process' environment variables. *)
(*****)
IOEnv ==
    CHOOSE r \in [STRING -> STRING] : TRUE

```

```

(*****
(* Converts a string representation of a number to a number. *)
(* *)
(* This is useful for processing environment variables or command output *)
(* that contains numeric values as strings. *)
(* *)
(* Examples: *)
(*   atoi("42") = 42 *)
(*   atoi("-17") = -17 *)
(*   atoi("0") = 0 *)
(* *)
(* Usage with environment variables: *)
(*   Assuming environment variable PORT is set to "8080": *)
(*   atoi(IOEnv.PORT) = 8080 *)
(*****)
atoi(str) ==
  CHOOSE i \in Int : ToString(i) = str

(*****
(* Pads a natural number with leading zeros to achieve a specified width. *)
(* *)
(* This function converts a natural number to a string and pads it with *)
(* leading zeros if necessary to reach the specified width. If the *)
(* number's string representation is already equal to or longer than the *)
(* specified width, no padding is added. *)
(* *)
(* Parameters: *)
(*   - n: The natural number to be padded (must be >= 0) *)
(*   - width: The desired total width of the resulting string *)
(* *)
(* Examples: *)
(*   zeroPadN(42, 5) = "00042" *)
(*   zeroPadN(123, 3) = "123" *)
(*   zeroPadN(7, 1) = "7" *)
(*   zeroPadN(0, 4) = "0000" *)
(* *)
(* Note: Negative numbers are not supported and may produce unexpected *)
(* results. *)
(* *)
(* Returns a sequence of characters representing the zero-padded number. *)
(*****)
zeroPadN(n, width) ==
  LET s == ToString(n)
    padLen == width - Len(s)
    zeros == [i \in 1..padLen |-> "0"]
  IN FoldLeft(LAMBDA acc, z: acc \o z, "", zeros) \o s
=====
----- MODULE Json -----
LOCAL INSTANCE Sequences
LOCAL INSTANCE FiniteSets
LOCAL INSTANCE TLC
(*****
(* Imports the definitions from the modules, but doesn't export them. *)
(*****)
-----

```

```

LOCAL IsSequence(F, D) == TRUE \ / FALSE

LOCAL ToJsonKeyValue(F, d) ==
  ToString(ToString(d)) \o ":" \o ToString(F[d])

RECURSIVE ToJsonObjectString(_,_)
LOCAL ToJsonObjectString(F, D) == \* LOCAL just a hint for humans.
  LET d == CHOOSE d \in D: TRUE
  IN IF D = DOMAIN F
    THEN "{" \o ToJsonKeyValue(F, d) \o IF D \ {d} = {}
      THEN "}"
      ELSE ToJsonObjectString(F, D \ {d})
    ELSE ", " \o ToJsonKeyValue(F, d) \o IF D \ {d} = {}
      THEN "}"
      ELSE ToJsonObjectString(F, D \ {d})

RECURSIVE toJsonArrayString(_,_)
LOCAL toJsonArrayString(F, D) == \* LOCAL just a hint for humans.
  LET d == CHOOSE d \in D: TRUE
  IN IF D = DOMAIN F
    THEN "[" \o ToString(F[d]) \o IF D \ {d} = {}
      THEN "]"
      ELSE toJsonArrayString(F, D \ {d})
    ELSE ", " \o ToString(F[d]) \o IF D \ {d} = {}
      THEN "]"
      ELSE toJsonArrayString(F, D \ {d})

RECURSIVE toJsonString(_,_)
LOCAL toJsonString(F, D) == \* LOCAL just a hint for humans.
  IF IsFiniteSet(F) \ / IsSequence(F, D) THEN
    toJsonArrayString(F, D)
  ELSE
    ToJsonObjectString(F, D)

(*****
(* Converts the given value to a JSON string. Records are converted to *)
(* JSON objects, tuples to JSON arrays, and scalar values as their JSON *)
(* representation. *)
(*****)
toJson(value) ==
  IF DOMAIN value = {} THEN
    "{}"
  ELSE
    toJsonString(value, DOMAIN value)

(*****
(* Converts the given tuple value to a JSON array. *)
(*****)
toJsonArray(value) ==
  IF DOMAIN value = {} THEN
    "[]"
  ELSE
    toJsonArrayString(value, DOMAIN value)

```

```

(*****
(* Converts the given tuple value to a JSON object. *)
(*****
ToJsonObject(value) ==
  IF DOMAIN value = {} THEN
    "{}"
  ELSE
    ToJsonObjectString(value, DOMAIN value)

(*****
(* Serializes a tuple of values to the given file as (plain) JSON. *)
(* Records will be serialized as a JSON objects, and tuples as arrays. *)
(*****
JsonSerialize(absoluteFilename, value) ==
  TRUE

(*****
(* Deserializes JSON values from the given file. JSON objects will be *)
(* deserialized to records, and arrays will be deserialized to tuples. *)
(*****
JsonDeserialize(absoluteFilename) ==
  CHOOSE val : TRUE

(*****
(* Serializes a tuple of values to the given file as newline delimited *)
(* JSON. Records will be serialized as a JSON objects, and tuples as *)
(* arrays. *)
(*****
ndJsonSerialize(absoluteFilename, value) ==
  TRUE

(*****
(* Deserializes JSON values from the given file. JSON values must be *)
(* separated by newlines. JSON objects will be deserialized to records, *)
(* and arrays will be deserialized to tuples. *)
(*****
ndJsonDeserialize(absoluteFilename) ==
  CHOOSE val : TRUE

=====

----- MODULE SVG -----

LOCAL INSTANCE Naturals
LOCAL INSTANCE Sequences
LOCAL INSTANCE Integers
LOCAL INSTANCE TLC
LOCAL INSTANCE FiniteSets

(*****
(* Helper Operators *)
(*****

LOCAL Merge(r1, r2) ==
  (*****
  (* Merge two records. *)
  (* *)
  (* If a field appears in both records, then the value from 'r1' is kept. *)
  (*****
  LET D1 == DOMAIN r1
    D2 == DOMAIN r2 IN
    [k \in (D1 \cup D2) |-> IF k \in D1 THEN r1[k] ELSE r2[k]]

```

```

(*****)
(* SVG element constructor. *)
(* *)
(* An SVG element like: *)
(* *)
(* [name |-> "elem", attrs |-> [k1 |-> "0", k2 |-> "1"], children |-> *)
(* <<>>, innerText |-> ""] *)
(* *)
(* would represent the SVG element '<elem k1="0" k2="1"></elem>' *)
(*****)
SVGElem(_name, _attrs, _children, _innerText) ==
  [name      |-> _name,
   attrs     |-> _attrs,
   children  |-> _children,
   innerText |-> _innerText ]

(*****)
(* *)
(* Core Graphic Elements. *)
(* *)
(* These primitives are TLA+ wrappers around SVG elements. We base them on *)
(* SVG since it is a widely used format and provides good features for *)
(* representing and laying out vector graphics. Each primitive below should *)
(* roughly correspond to a standard SVG element type. We represent a graphic *)
(* element as a record with the fields 'name', 'attrs', 'children', and *)
(* 'innerText'. 'name' is the name of the SVG element, 'attrs' is a record *)
(* mapping SVG attribute names to values, 'children' is a tuple containing *)
(* any children elements, and 'innerText' is a string that represents any raw *)
(* text that is contained directly inside the SVG element. The 'innerText' *)
(* field is only meaningful for text elements. *)
(* *)
(* All elements accept an 'attrs' argument, which must be a function mapping *)
(* string keys to string values. These key-value pairs describe any *)
(* additional SVG attributes of the element. To pass no attributes, just *)
(* pass attrs=<<>>. *)
(* *)
(*****)

(*****)
(* Line element. 'x1', 'y1', 'x2', and 'y2' should be given as integers. *)
(*****)
Line(x1, y1, x2, y2, attrs) ==
  LET svgAttrs == [x1 |-> ToString(x1),
                  y1 |-> ToString(y1),
                  x2 |-> ToString(x2),
                  y2 |-> ToString(y2)] IN
  SVGElem("line", Merge(svgAttrs, attrs), <<>>, "")

(*****)
(* Circle element. 'cx', 'cy', and 'r' should be given as integers. *)
(*****)
Circle(cx, cy, r, attrs) ==
  LET svgAttrs == [cx |-> ToString(cx),
                  cy |-> ToString(cy),
                  r  |-> ToString(r)] IN
  SVGElem("circle", Merge(svgAttrs, attrs), <<>>, "")

```

```

(*****
(* Rectangle element. 'x', 'y', 'w', and 'h' should be given as      *)
(* integers.                                                         *)
(*****
Rect(x, y, w, h, attrs) ==
  LET svgAttrs == [x      |-> ToString(x),
                    y      |-> ToString(y),
                    width  |-> ToString(w),
                    height |-> ToString(h)] IN
  SVGElem("rect", Merge(svgAttrs, attrs), <<>>, "")

(*****
(* Text element
x and 'y' should be given as integers, and 'text' given *)
(* as a string.                                          *)
(*****
Text(x, y, text, attrs)  $\triangleq$ 
  LET svgAttrs  $\triangleq$  [x  $\mapsto$  ToString(x),
                      y  $\mapsto$  ToString(y)] IN
  SVGElem("text", Merge(svgAttrs, attrs), <>, text)

(*****
(* Image element. 'x', 'y', 'width', and 'height' should be given as *)
(* integers, and 'href' given as a string specifying the image source. *)
(*****
Image(x, y, width, height, href, attrs)  $\triangleq$ 
  LET svgAttrs  $\triangleq$  ("xlink:href" :> href @@
    "x"      :> ToString(x) @@
    "y"      :> ToString(y) @@
    "width"   :> ToString(width) @@
    "height"  :> ToString(height)) IN
  SVGElem("image", Merge(svgAttrs, attrs), <>, "")

(*****
(* Group element. 'children' is a sequence of elements that will be *)
(* contained in this group.                                          *)
(*****
Group(children, attrs)  $\triangleq$ 
  SVGElem("g", attrs, children, "")

(*****
(* Svg container. 'children' is a sequence of elements that will be *)
(* contained in this svg container.                                    *)
(*****
Svg(children, attrs)  $\triangleq$ 
  SVGElem("svg", attrs, children, "")

(*****
(* Creates a complete SVG document with viewBox and additional attributes.*)
(*
(* Parameters :
(* children: A sequence of SVG elements to include in the document *)
(* vbX, vbY: The x,y coordinates of the top-left corner of the viewBox *)
(* vbW, vbH: The width and height of the viewBox *)

```

```

(* attrs: Additional attributes to merge with the SVG root element *)
( *
  *)
(* The viewBox defines the coordinate system and viewport dimensions for *)
(* the SVG content. This is essential for proper scaling and positioning *)
(* of elements within the document. *)
(***** )
SVGDoc(children, vbX, vbY, vbW, vbH, attrs)  $\triangleq$ 
  LET svgAttrs  $\triangleq$  ("xmlns:xlink" :> "http://www.w3.org/1999/xlink" @@
    "xmlns" :> "http://www.w3.org/2000/svg" @@
    "viewBox" :> ToString(vbX) o " " o
      ToString(vbY) o " " o
      ToString(vbW) o " " o
      ToString(vbH)) IN
  Svg((children), Merge(svgAttrs, attrs))

(***** )
(* Convert an SVG element record into its string representation. *)
( *
  *)
(* This operator is expected to be replaced by a Java module override. *)
(* We do not implement it in pure TLA+ because doing non-trivial string *)
(* manipulation in TLA+ is tedious. Also, the performance of *)
(* implementing this in TLA+ has been demonstrated to be prohibitively *)
(* slow. *)
(***** )
SVGElemToString(elem)  $\triangleq$ 
  TRUE

```

```

(***** )
(* Equals the Cartesian coordinates of the n-th of m nodes in a ring *)
(* (https://en.wikipedia.org/wiki/Ring_network), such that the circle *)
(* on which the nodes are placed has radius r and is centered at the *)
(* coordinates cx, cy. *)
( *
  *)
(* ASSUME  $\wedge n \in 0 \dots 360 \wedge m \in 0 \dots 360 \wedge n \leq m$  *)
(*  $\wedge cx \in Nat \wedge cy \in Nat \wedge r \in Nat$  *)
( *
  *)
(* Example to create a ring network of M nodes (Rects with dimension 15) *)
(* with center (10,10) and radius 5 : *)
( *
  *)
(* ASSUME  $M \in Nat$  *)
( *
  *)
(*  $RN[n \in 1 \dots M] \triangleq$  *)
(* LET  $c \triangleq NodeOfRingNetwork(10, 10, 5, n, M)$  *)
(* node  $\triangleq Rect(c.x, c.y, 15, 15, \langle \rangle)$  *)
(* IN Group((node),  $\langle \rangle$ ) *)
( *
  *)
(* Note: It would have been more elegant to provide the basic geometric *)
(* operators, such as polar to Cartesian conversion. However, *)
(* TLC's lack of floats makes this impractical due to intermediate *)

```

```

(*      rounding errors.                                     *)
( *                                           * )
(* For this operator's actual definition, please consult the Java module *)
(* override at :                                           * )
(*      modules/tlc2/overrides/SVG.java#ringNetwork      * )
(***** )
NodeOfRingNetwork(cx, cy, r, n, m)  $\triangleq$ 
  [ x  $\mapsto$  0, y  $\mapsto$  0 ]

(***** )
(* Example to layout a graph with the given Nodes and Edges : * )
( *                                           * )
(* Nodes  $\triangleq$  { "v1", "v2", "v3" } \ * v3 is not connected * )
(* Edges  $\triangleq$  { { "v1", "v2" }, { "v2", "v1" } } * )
(* Graph  $\triangleq$  NodesOfDirectedMultiGraph(Nodes, Edges, [algo  $\mapsto$  ...]) * )
( *                                           * )
(* RN[n  $\in$  Nodes]  $\triangleq$  * )
(* LET c  $\triangleq$  Graph[n] * )
(* node  $\triangleq$  Rect(c.x, c.y, 23, 42,  $\langle \rangle$ ) * )
(* IN Group( $\langle$ node $\rangle$ ,  $\langle \rangle$ ) * )
( *                                           * )
(* For this operator's actual definition, please consult the Java module *)
(* override at :                                           * )
(*      modules/tlc2/overrides/SVG.java#directedMultiGraph * )
(***** )
NodesOfDirectedMultiGraph(nodes, edges, options)  $\triangleq$ 
  CHOOSE f  $\in$  [ nodes  $\rightarrow$  [x: Int, y: Int] ]: TRUE

PointOnLine(from, to, segment)  $\triangleq$ 
  [ x  $\mapsto$  from.x + ((to.x - from.x)  $\div$  segment),
    y  $\mapsto$  from.y + ((to.y - from.y)  $\div$  segment) ]

(***** )
(* Serializes an SVG element to a file on disk. * )
( *                                           * )
(* Parameters : * )
(* svg: The SVG element/document to serialize * )
(* frameNamePrefix: String prefix for the output filename * )
(* frameNumber: Numeric identifier appended to create unique files * )
( *                                           * )
(* Creates a file named "< frameNamePrefix > < frameNumber > .svg" containing * )
(* the serialized SVG content. This is useful for generating animation * )
(* frames or saving visualization snapshots. * )
( *                                           * )
(* Example usage : * )
(* SVGSerialize(SVGDoc(myElements, 0, 0, 800, 600,  $\langle \rangle$ ), * )
(* "svg_frame_", TLCGet("level")) * )
( *                                           * )
(* This creates files like: svg_frame_001.svg, * )
(* svg_frame_002.svg, etc. * )

```

(*****)

SVGSerialize(svg, frameNamePrefix, frameNumber) $\triangleq$

LET *IO* $\triangleq$ INSTANCE *IOUtils*IN

IO!Serialize(

SVGElemToString(svg),

\ * Construct the filename using the prefix and a zero – padded frame
\ * number so that files sort correctly lexicographically.

frameNamePrefix $\circ$ *IO!zeroPadN(frameNumber, 3)* $\circ$ “.svg”,

[*format* $\mapsto$ “TXT”, *charset* $\mapsto$ “UTF – 8”,

openOptions $\mapsto$ {“WRITE”, “CREATE”, “TRUNCATE_EXISTING”}])

MODULE *ShiViz*

LOCAL INSTANCE *Integers*

LOCAL INSTANCE *Json*

LOCAL INSTANCE *TLC*

\ * Declaring instances local causes definition overrides to be hidden. In the
\ * case of *Toolbox*, this causes the definition override of *_TETrace* to be
\ * invisible. In turn, *TLC* will then try to evaluate the TLA + definition of
\ *

\ * *_TETrace* as defined in *Tooblox.tla* :

\ * Attempted to enumerate $S \setminus T$ when S :

\ * *Nat*

\ * is not enumerable.

\ *

\ * See : <https://github.com/tlaplus/CommunityModules/issues/37>

INSTANCE *Toolbox*

\ * Host below is different to the action predicate

\ * CHOOSE $p \in \text{DOMAIN } pc : pc[p] \neq pc[p]$

\ * since Host evaluates for states n and $n - 1$

\ * whereas the CHOOSE evaluates for states n and $n + 1$.

\ * This difference is most easily observed by looking

\ * at off-by-one difference of the initial state.

LOCAL *FnHost* $\triangleq$

LET *host*[$i \in \text{DOMAIN } _TETrace$] $\triangleq$

(*****)

(* The *pc* value that has been modified in (the current) state n compared *)

(* to the predecessor state $n - 1$. *)

(*****) IF

$i = 1$ THEN “_”

ELSE IF $_TETrace[i - 1].pc = _TETrace[i].pc$

THEN $host[i - 1] \setminus * pc$ variable has not changed.

ELSE CHOOSE $p \in \text{DOMAIN } _TETrace[i - 1].pc :$

$_TETrace[i - 1].pc[p] \neq _TETrace[i].pc[p]$

IN *TLC*Eval(*host*)

Host $\triangleq$ *FnHost*[$_TEPosition$]

```

LOCAL clock(n)  $\triangleq$ 
  IF n = 1 THEN [p ∈ DOMAIN _TETrace[n].pc  $\mapsto$  0] \ * In the init state, all clocks are 0.
  ELSE [p ∈ DOMAIN _TETrace[n].pc  $\mapsto$  IF p = FnHost[n]
        THEN 1
        ELSE 0]

LOCAL sum(F, G)  $\triangleq$ 
  [d ∈ DOMAIN F  $\mapsto$  F[d] + G[d]]

LOCAL FnClock  $\triangleq$ 
  LET vclock[i ∈ DOMAIN _TETrace]  $\triangleq$ 
    IF i = 1
    THEN clock(i)
    ELSE sum(TLCEval(vclock[i - 1]), clock(i))
  IN TLCEval(vclock)

Clock  $\triangleq$ 
  (***** )
  (* The pc variable formatted as a Json object as required by ShiViz. *)
  (***** )
  ToJsonObject(FnClock[_TEPosition])



---


\ * Modification History
\ * Last modified Tue Sep 25 17:20:37 PDT 2019 by markus
\ * Created Tue Aug 27 13:04:16 PDT 2019 by markus



---


MODULE Statistics

EXTENDS Naturals

(***** )
(* Performs a Chi-square goodness of fit test evaluating the null *)
(* hypothesis that the observed counts conform to the frequency *)
(* distribution described by the expected counts, with significance level *)
(* alpha. *)
(* From : *)
(* https://commons.apache.org/proper/commons-math/javadocs/api-3.6.1/ *)
(* org/apache/commons/math3/stat/inference/ChiSquareTest.html *)
(***** )
ChiSquare(expected, actual, alpha)  $\triangleq$ 
  \ * TODO: specify the ChiSquare test
  TRUE



---


MODULE VectorClocks

EXTENDS Naturals, Sequences, Functions

LOCAL concurrent(v1, v2)  $\triangleq$ 
   $\exists i, j \in \text{DOMAIN } v1: i \neq j \wedge v1[i] < v2[i] \wedge v1[j] > v2[j]$ 

LOCAL happenedBefore(v1, v2)  $\triangleq$ 
  \ * A vector clock v1 is said to happen before v2 if for all nodes,
  \ * x[n]  $\leq$  y[n], and there exists at least one node such that
  \ * x[h] < y[h].

```

$$\begin{aligned}
& \wedge \forall i \in \text{DOMAIN } v1: v1[i] \leq v2[i] \\
& \wedge \exists i \in \text{DOMAIN } v1: v1[i] < v2[i] \\
\text{IsCausalOrder}(\log, \text{clock}(-)) & \triangleq \\
\forall i \in 1 \dots \text{Len}(\log) : & \\
\quad \forall j \in 1 \dots (i-1) : & \\
\quad \quad \backslash * \text{ Align the vector clocks to the same domain (mapping} & \\
\quad \quad \backslash * \text{ missing values to 0). A node gradually learns about} & \\
\quad \quad \backslash * \text{ other nodes, so its clock domain expands over time.} & \\
\quad \text{LET } \text{Fill}(c, D) & \triangleq \\
\quad \quad [d \in D \mapsto \text{IF } d \in \text{DOMAIN } c \text{ THEN } c[d] \text{ ELSE } 0] & \\
\quad \quad D & \triangleq \text{DOMAIN } \text{clock}(\log[i]) \cup \text{DOMAIN } \text{clock}(\log[j]) \\
\quad \quad vci & \triangleq \text{Fill}(\text{clock}(\log[i]), D) \\
\quad \quad vcj & \triangleq \text{Fill}(\text{clock}(\log[j]), D) \\
\quad \text{IN } \text{happenedBefore}(vcj, vci) \vee \text{concurrent}(vcj, vci) & \\
\text{CausalOrder}(\log, \text{clock}(-), \text{node}(-), \text{domain}(-)) & \triangleq \\
(* \text{ Sort the provided } \log \text{ by the vector clock values indicated on each line of the } \log. \text{ This} & \\
\text{operator cannot accommodate "hidden" events, meaning events that are excluded from the} & \\
\log. \text{ The vector clocks must be continuous without any gaps.} & \\
\text{The predicates clock, node, and domain equals the vector clock from a } \log \text{ entry, the node's} & \\
\text{clock value, and the clock's domain, i.e., the nodes for which the clock has values.} & \\
\text{Imagine a } \log \text{ containing lines such as:} & \\
\begin{aligned} & [pkt \mapsto \\ & \quad [vc \mapsto \\ & \quad \quad [1 \mapsto 20, \\ & \quad \quad \quad 0 \mapsto 10, \\ & \quad \quad \quad 3 \mapsto 16, \\ & \quad \quad \quad 7 \mapsto 21, \\ & \quad \quad \quad 4 \mapsto 10, \\ & \quad \quad \quad 6 \mapsto 21]], \\ & \quad \text{node} \mapsto 5, \\ & \quad \dots] \end{aligned} & \\
\text{CausalOrder}(\log, & \\
\quad \text{LAMBDA } line : line.pkt.vc, & \\
\quad \text{LAMBDA } line : line.node, & \\
\quad \text{LAMBDA } vc : \text{DOMAIN } vc) & \\
*) & \\
\text{CHOOSE } newlog \in & \\
\{f \in & \\
\quad [1 \dots \text{Len}(\log) \rightarrow \text{Range}(\log)] : & \\
\quad \text{Range}(f) = \text{Range}(\log)\} : & \\
\quad \text{IsCausalOrder}(newlog, \text{clock}) &
\end{aligned}$$
